

# Mobile Development :

## ***5 : Loading Data Asynchronously Using Future Builder***



**Professor Imed Bouchrika**

National School of Artificial Intelligence  
imed.bouchrika@ensia.edu.dz

# Outline :

- **Section 1 : Revision**
  - *Structuring & Organization*
  - *Form Validation*
- **Section 2 : API Endpoint**
  - *Skeleton Backend APIs using Flask/Vercel*
- **Section 3 : Async, Timer & Future Builder**
  - *Writing Async Functions*
  - *Future Builder Widget*
  - *Refresh on slide*
  - *Timer for Periodic Events*
  - *Incrementing App*

# Section 1

**Structuring and Organizing..**

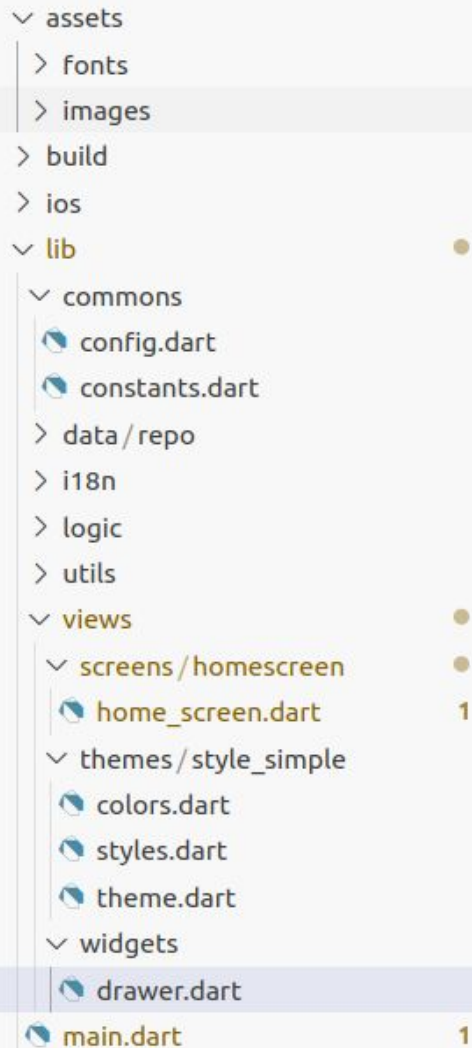




# Project Organization

# Structuring and Organizing Mobile Projects

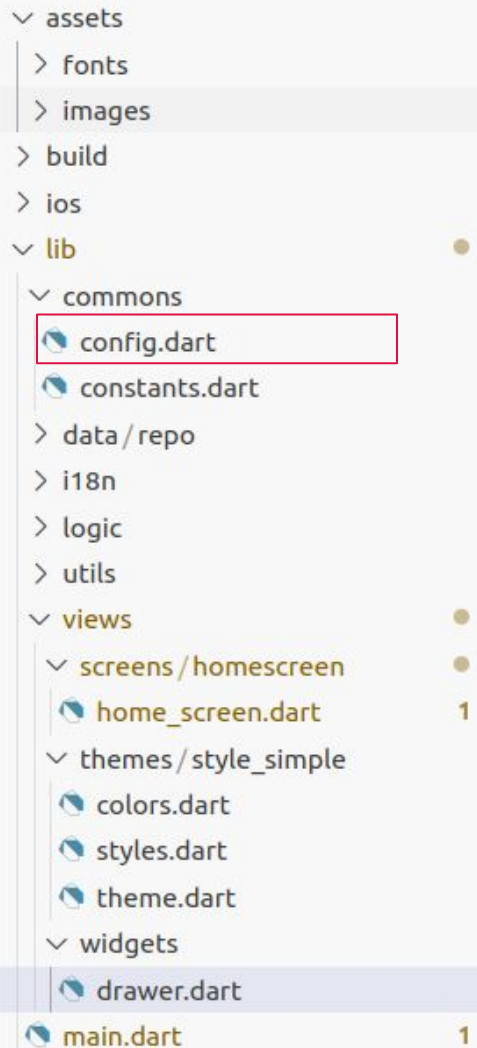
- Project Structure :



# Structuring and Organizing Mobile Projects

- **config.dart:**

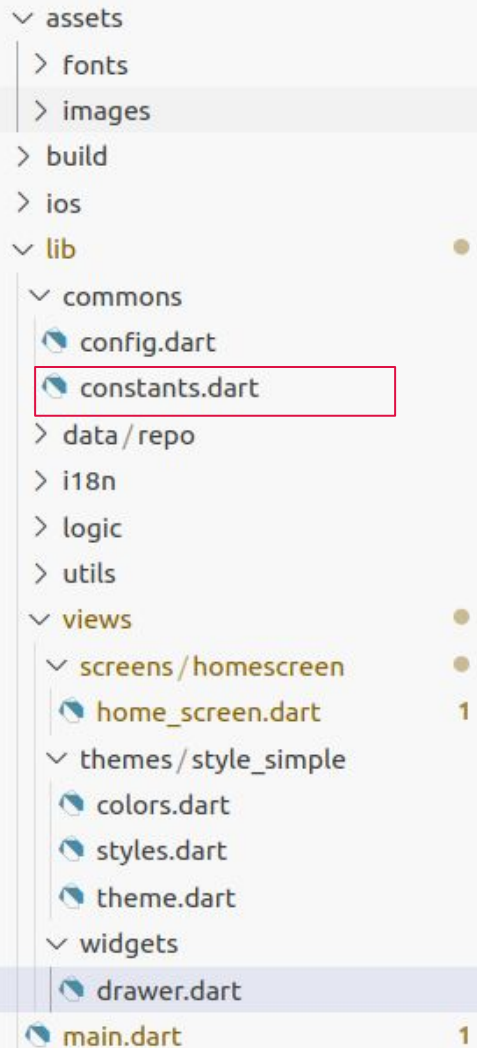
```
const APP_NAME = "Incrementer App";  
const APP_VERSION = "19.12.1";
```



# Structuring and Organizing Mobile Projects

- **constants.dart:**

```
const INCREMENT_STEP = 1;
```

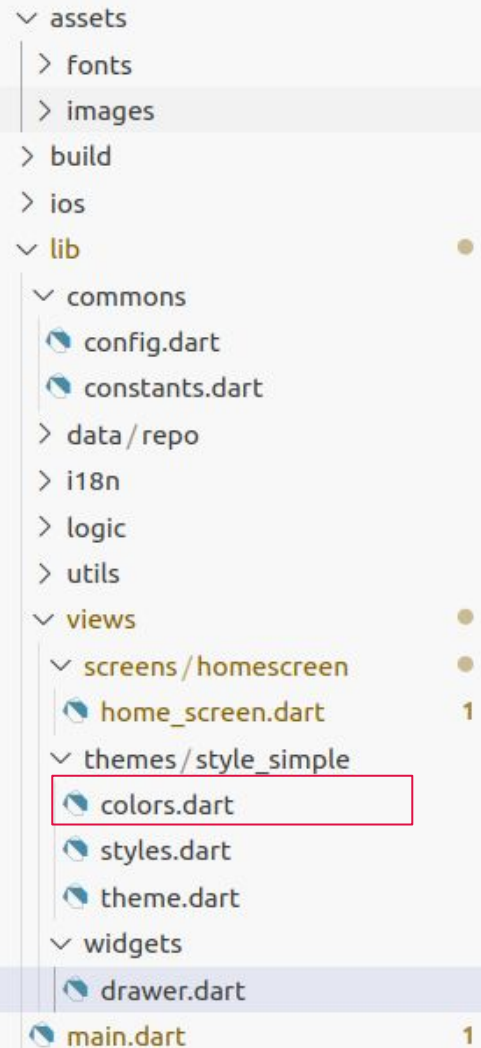


# Structuring and Organizing Mobile Projects

- Themes / Style\_Simple / Colors.dart

```
import 'package:flutter/material.dart';

const backColor = Color(0xffeff9ff);
const blackColor = Color(0xff000000);
const whiteColor = Color.fromARGB(255, 248, 241, 241);
const white1Color = Color(0xffff2f2f2);
const textColor = Color(0xff1e2123);
const text1Color = Color(0xff817f8c);
...
```

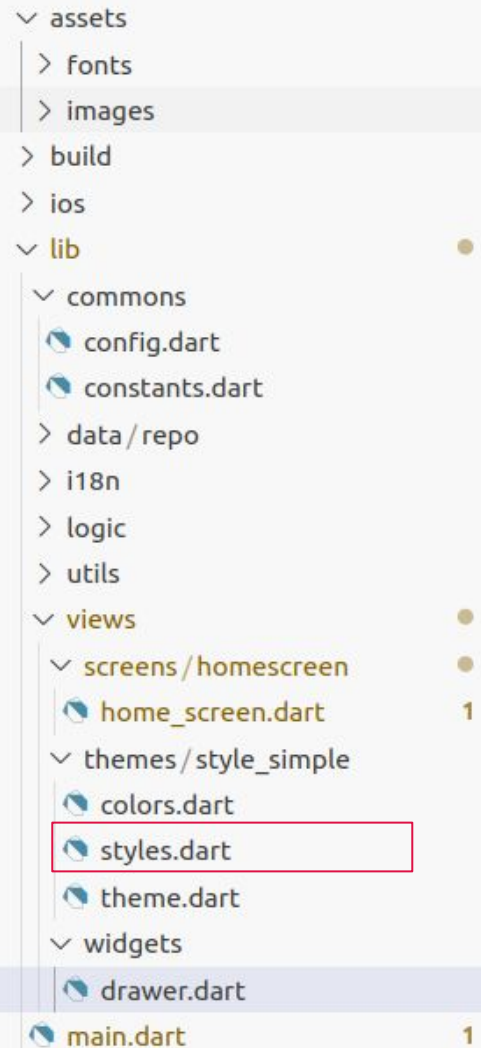




# Structuring and Organizing Mobile Projects

- Themes / Style\_Simple / Styles.dart

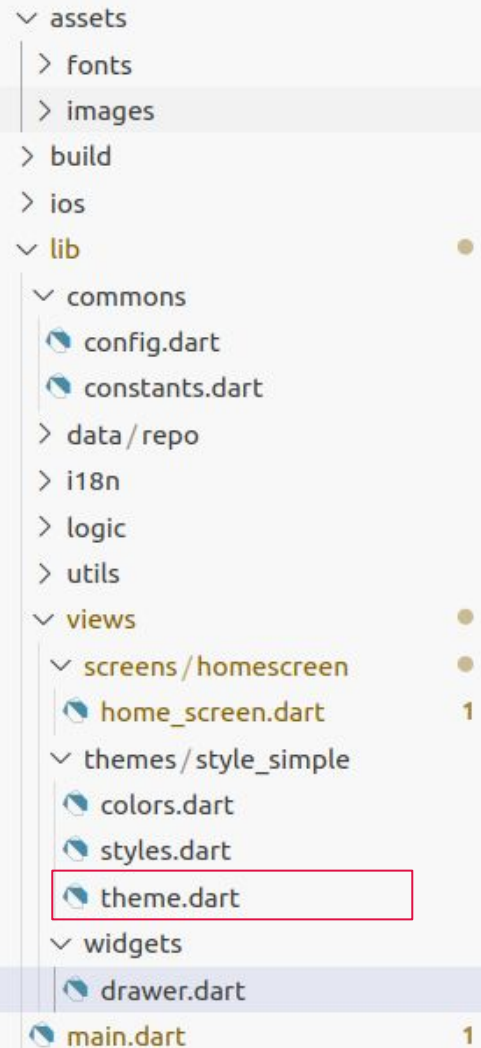
```
const bartitleTextStyle = TextStyle(  
  fontSize: 20.0,  
  color: barTitleColor,  
  fontFamily: "Satoshi",  
  fontWeight: FontWeight.w700,  
  fontStyle: FontStyle.normal,  
);  
  
const button1TextStyle = TextStyle(  
  fontSize: 18.0,  
  color: whiteColor,  
  fontFamily: "Satoshi",  
  fontWeight: FontWeight.w600,  
  fontStyle: FontStyle.normal,  
);
```



# Structuring and Organizing Mobile Projects

- Themes / Style\_Simple / Themes.dart

```
final THEME_DATA = ThemeData(  
  useMaterial3: true,  
  primaryColor: backColor,  
  scaffoldBackgroundColor: whiteColor,  
  drawerTheme: const DrawerThemeData(  
    shape: BeveledRectangleBorder(  
      borderRadius: BorderRadius.zero,  
    ),  
  ),  
);
```



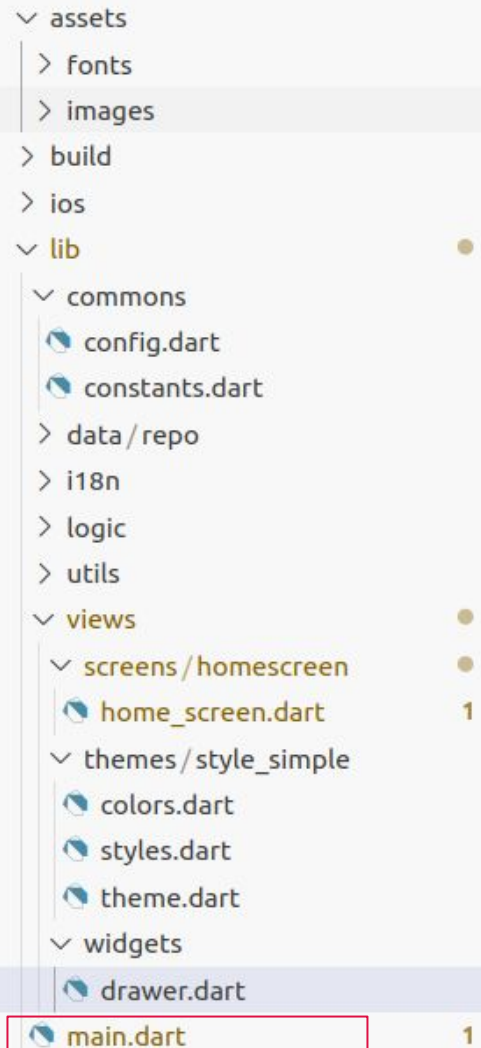
# Structuring and Organizing Mobile Projects

- Main.dart

```
Future<bool> my_init_app() async {
  //some initialization here...
  return true;
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await my_init_app();
  runApp(MainApp());
}

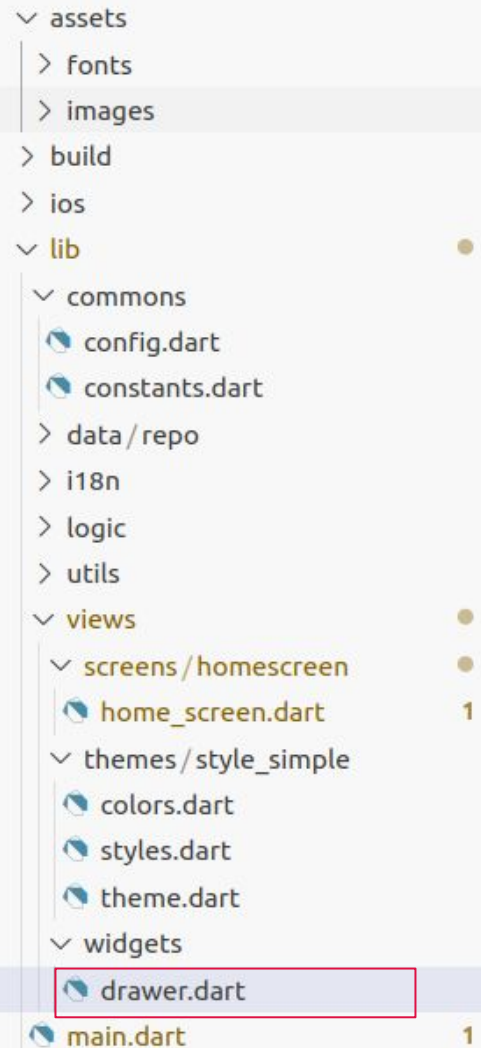
class MainApp extends StatelessWidget {
  const MainApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: APP_NAME,
      theme: THEME_DATA,
      home: HomeScreen(),
    );
  }
}
```



# Structuring and Organizing Mobile Projects

- Drawer

```
class MySideBarDrawer extends StatelessWidget {  
  const MySideBarDrawer({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Drawer(  
      child: ListView(padding: EdgeInsets.zero,  
        children: [  
          ListTile(  
            title: const Text("Home"),  
            leading: const Icon(Icons.home_filled),  
            onTap: () =>  
              Navigator.pushNamed(context,  
                'SOME_SCREEN_ADDRESS', arguments: 0),  
          ),  
        ])),  
    );  
  }  
}
```

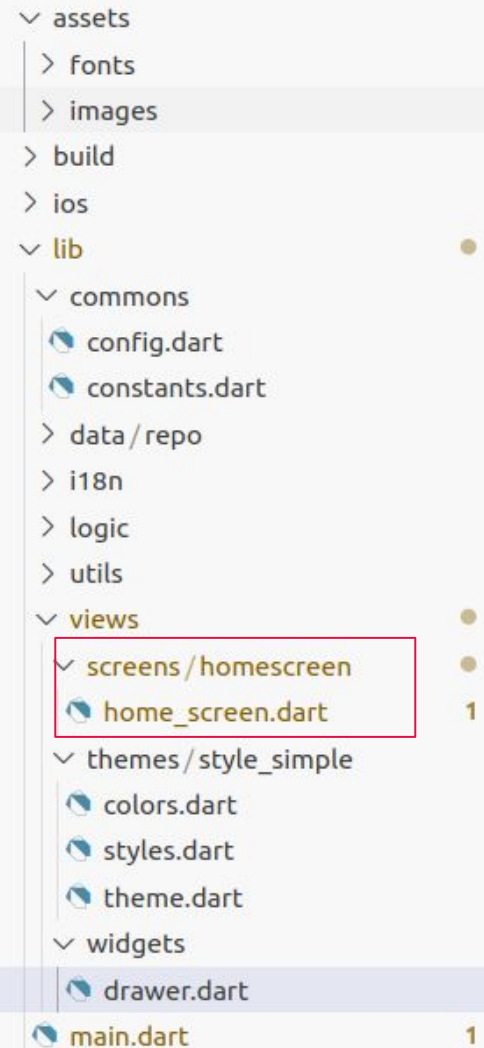


```

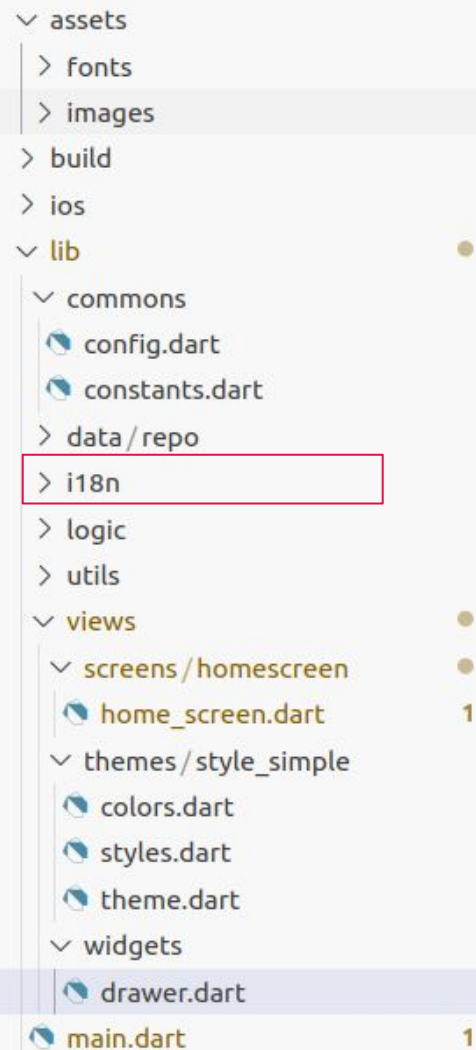
class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});
  @override
  State<HomeScreen> createState () => _HomeScreenState ();
}

class _HomeScreenState extends State<HomeScreen> {
  int number = 1;
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Welcome ENSIA')),
      drawer: MySideBarDrawer(),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          crossAxisAlignment: CrossAxisAlignment.center,
          children: [
            Text('Click to Increment', style: startTextStyle),
            SizedBox(height: 20),
            Text('$number'),
            SizedBox(height: 20),
            ElevatedButton(
              onPressed: () {
                number = number + 1;
                setState(() {});
              },
              style: ElevatedButton.styleFrom(backgroundColor: Colors.purple),
              child: Text('Increment', style: button1TextStyle),
            ),
          ],
        ),
      ),
    );
  }
}

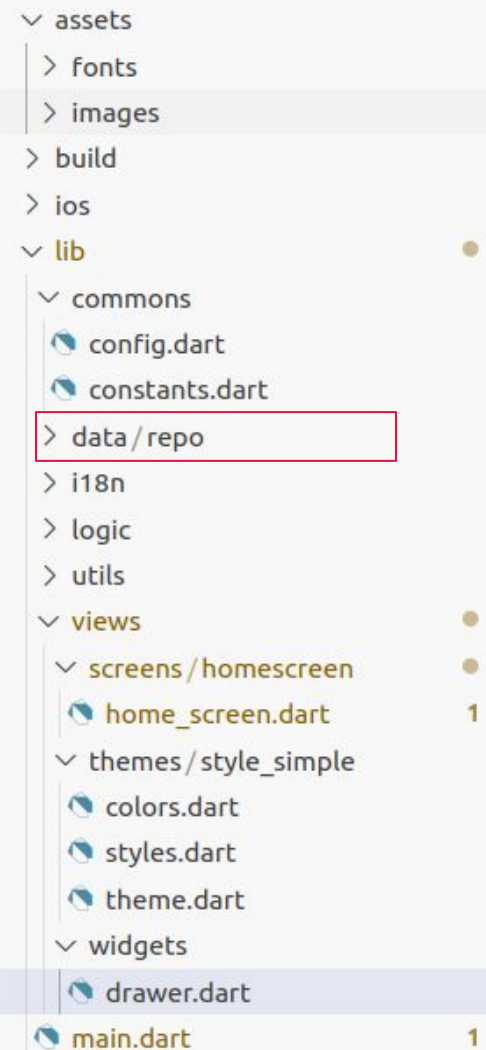
```



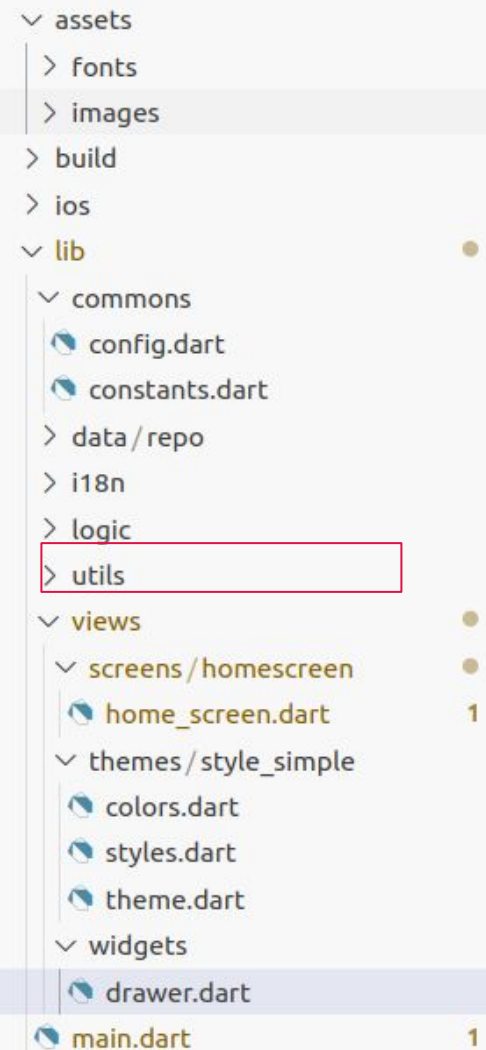
# Structuring and Organizing Mobile Projects



# Structuring and Organizing Mobile Projects



# Structuring and Organizing Mobile Projects





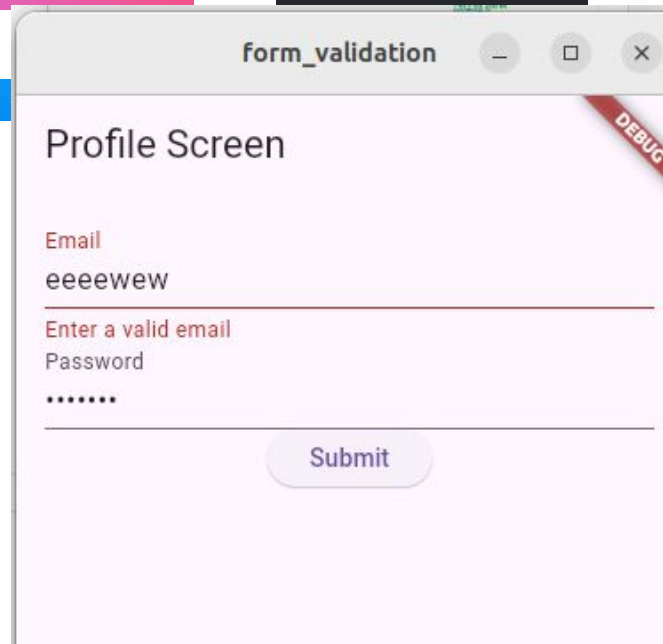


# Flutter Form Validation

# Validating Form Inputs in Flutter

- **Showing Errors next to form fields**
  - You can code it manually from scratch
  - Or
    - Use Form Flutter Widget
    - Input Widgets have the attribute:

```
validator: _validateEmail
```



The screenshot shows a mobile application window titled "form\_validation". Inside, there is a "Profile Screen" with two form fields. The "Email" field contains the text "eeeeew" and has a red error message "Enter a valid email" displayed below it. The "Password" field contains a masked password ".....". A "Submit" button is located at the bottom right of the form. A red "DEBUG" banner is visible in the top right corner of the application window.

```

class ProfileScreen extends StatefulWidget {
  const ProfileScreen ({super.key});
  @override
  State<ProfileScreen> createState () => _ProfileScreenState ();
}

class ProfileScreenState extends State<ProfileScreen> {
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController ();
  final _passwordController = TextEditingController ();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Profile Screen')),
      body: Form(
        key: _formKey,
        child: Container(
          margin: const EdgeInsets.all(16.0),
          child: Column(
            children: [
              TextFormField(
                controller: _emailController,
                decoration: InputDecoration(labelText: 'Email'),
                validator: _validateEmail,
              ),
              TextFormField(
                controller: _passwordController,
                decoration: InputDecoration(labelText: 'Password'),
                obscureText: true,
                validator: (value) {
                  if (value == null || value.isEmpty) {
                    return 'Please enter your password' ;
                  }
                  if (value.length < 6) {

```

```
class ProfileScreen extends StatefulWidget {
  const ProfileScreen ({super.key});
  @override
  State<ProfileScreen> createState () => _ProfileScreenState ();
}
```

```
class ProfileScreenState extends State<ProfileScreen> {
```

```
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return Scaffold(
```

```
      appBar: AppBar(title: Text('Profile')),
```

```
      body: Form(
```

```
        key: _formKey,
```

```
        child: Container(
```

```
          margin: const EdgeInsets.all(10),
```

```
          child: Column(
```

```
            children: [
```

```
              TextFormField(
```

```
                controller: _emailController,
```

```
                decoration: InputDecoration(labelText: 'Email'),
```

```
                validator: _validateEmail,
```

```
              ),
```

```
              TextFormField(
```

```
                controller: _passwordController,
```

```
                decoration: InputDecoration(labelText: 'Password'),
```

```
                obscureText: true,
```

```
                validator: (value) {
```

```
                  if (value == null || value.isEmpty) {
```

```
                    return 'Please enter your password' ;
```

```
                  }
```

```
                  if (value.length < 6) {
```

```
String? _validateEmail(String? value) {
```

```
  if (value == null || value.isEmpty) {
```

```
    return 'Email cannot be empty';
```

```
  } else if (!RegExp(r'^[\w-\.]@([\w-]+\.)+[\w-]{2,4}$').hasMatch(value)) {
```

```
    return 'Enter a valid email';
```

```
  } else {
```

```
    return null;
```

```
  }
```

```
}
```

```

class ProfileScreen extends StatefulWidget {
  const ProfileScreen ({super.key});
  @override
  State<ProfileScreen> createState () => _ProfileScreenState ();
}

class ProfileScreenState extends State<ProfileScreen> {
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController ();
  final _passwordController = TextEditingController ();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Profile Screen')),
      body: Form(
        key: _formKey,
        child: Container(

          ...

          ElevatedButton (
            onPressed: () {
              if (_formKey.currentState!.validate()) {
                // Form is valid, process data
                ScaffoldMessenger.of(
                  context,
                ).showSnackBar(SnackBar(content: Text('Processing Data')));
              }
            },
            child: Text('Submit'),
          ),
        ),
      ),
    ),
  ),
)

```

# Section 2

**Skeleton Backend using  
Serverless Technologies**

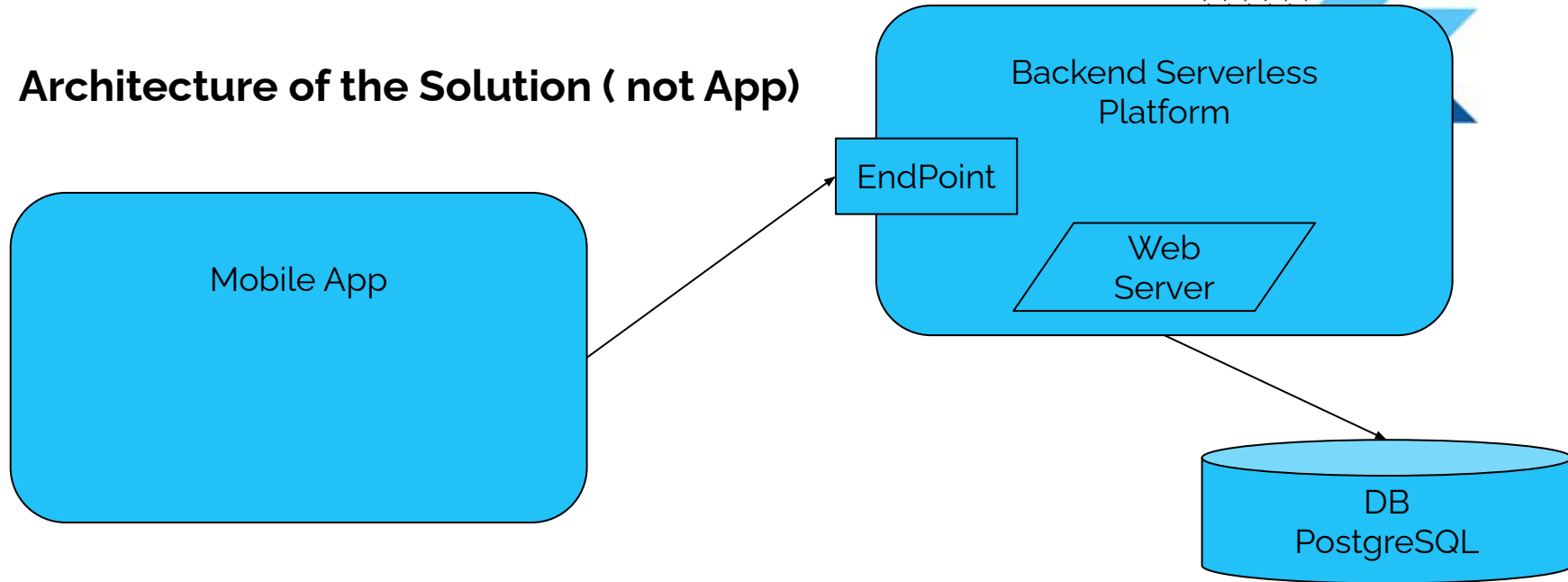
# Serverless Technology for Mobile App Backends



- **Improvement to the previous implementation:**
  - Use Python for Backend instead :
    - Existing Framework : **Flask** ( Or even Django )
    - Link with PostgreSQL provided by Supabase
  - Use Better Storage Facilities : S3 Storage or Firebase Storage
  - Integrate Firebase Services
  - Users' Authentication.

# Serverless Technology for Mobile App Backends

- Architecture of the Solution ( not App)





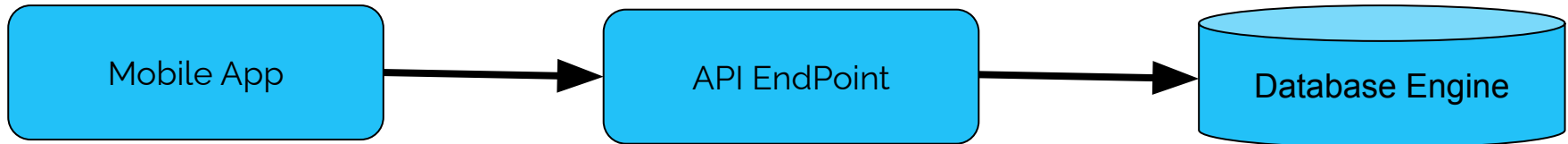
# Serverless Technology for Mobile App Backends

- **Architecture of the Solution ( not App ) : which one is better**

- **Direct Database Access**



- **Access to Database via EndPoint API**



# Serverless Technology for Mobile App Backends



- **Hello World in Flask**

- Fask, Like Java Spark, it is a **Python** micro framework for writing web applications.
- Microframe does usually not use contain common libraries like database access/ORM, form validation...
- It is written in Python, therefore you can important any python library you want from Machine Learning to other advanced libraries .
- For a full framework :
  - Python ⇒ Django
  - Java ⇒ SpringBoot

# Serverless Technology for Mobile App Backends

- Hello World in Flask

```
from flask import Flask
import json

app = Flask(__name__)

@app.route('/categories.get')
def get_categories():
    data=[{'id':1,'name':'Food'},{'id':2,'name':'Beverage'}]
    return json.dumps(data)

@app.route('/')
def index():
    return 'Welcome ENSIA Students from Flask!'

if __name__ == "__main__":
    app.run(port=8080)
```

# Serverless Technology for Mobile App Backends



- **Running the Hello World in Flask**
  - Install the required Python packages
    - `pip3 install flask`
    - `pip3 install waitress`
  - Run the Flask Application ( File name is : `hello.py` )
    - `python3 hello.py`
      - Or sometimes
    - `flask --app hello run`

# Serverless Technology for Mobile App Backends

- Hello World in Flask

```
from flask import Flask
import json

app = Flask(__name__)

@app.route('/categories.get')
def get_categories():
    data=[{'id':1, 'name': 'Food'}, {'id':2, 'name': 'Beverage'}]
    return json.dumps(data)

@app.route('/')
def index():
    return 'Welcome ENSIA Students from Flask!'

if __name__ == "__main__":
    app.run(port=8080)
```

This is the route which **binds URL** to a function to perform a given business logic

`__name__` is a special variable when the script is invoked directly (not imported), the `__name__` is set as `"__main__"`

# Serverless Technology for Mobile App Backends

- Getting Variables from URLs

```
from flask import Flask
import json

app = Flask(__name__)

@app.route('/categories.getById/<int:cat_id>')
def get_category_byId(cat_id):
    // some business logic here...
    return json.dumps(data)

@app.route('/')
def index():
    return 'Welcome ENSIA Students from Flask!'

if __name__ == "__main__":
    app.run(port=8080)
```

You can enclose URL variables inside  
**<type:variable\_name>** or simply  
**<variable\_name>**

# Serverless Technology for Mobile App Backends

- Getting GET/POST Variables

```
from flask import Flask , request
import json

app = Flask(__name__)

@app.route('/user.login', methods=['GET', 'POST'] )
def users_login():
    username=request.form.get('username')
    password=request.form.get('password')
    //some more business logic
    return json.dumps(data)

@app.route('/')
def index():
    return 'Welcome ENSIA Students from Flask!'

if __name__ == "__main__":
    app.run(port=8080)
```

GET or POST variables can be accessed  
via the special variable :

**request.form.get(VAR\_NAME)**

# Serverless Technology for Mobile App Backends

- Getting Binary Files from Uploads

```
from flask import Flask
import json

app = Flask(__name__)

@app.route('/photo.upload', methods=['POST'])
def users_login():
    if request.method=='POST':
        file=request.files.get('file_name')

        //some more business logic
        return json.dumps(data)

@app.route('/')
def index():
    return 'Welcome ENSIA Students from Flask!'

if __name__ == "__main__":
    app.run(port=8080)
```

Binary Data Files can be accessed via the special variable :

**request.files.get(FILE\_NAME)**



# Serverless Technology for Mobile App Backends

- Logging

```
from flask import Flask
import json

app = Flask(__name__)

app.logger.debug('A value for debugging')
app.logger.warning('A warning occurred (%d apples)', 42)
app.logger.error('An error occurred')
```

# Serverless Technology for Mobile App Backends



- **How to run it : Server-Centric approach**
  - Get a dedicated Server or even virtual ...
  - You need to manage it yourself
    - Upgrades
    - Security
  - Scaling, can be difficult to main
  - But, you have control over everything.
  - Cost : Cannot say ( it may cost \$5/month ..)

# Serverless Technology for Mobile App Backends



- **Introduction to Serverless Technologies**

- Even though, it is called serverless, the server is always there but:
  - Provided when there is a request from a user
  - You pay usually per request / processing duration / bandwidth transferred.
- Serverless Technologies provided by the hosting provider where they have the infrastructure to deploy dynamically servers/frameworks on demand.

# Serverless Technology for Mobile App Backends



- **Introduction to Serverless Technologies**

- **Benefits..**

- Maintenance of the technology taken care by the infrastructure provider
    - Scaling is always being taken care of. When more users, the hosting provider will deploy more resources dynamically, (but be prepared to pay more)

- **Drawbacks..**

- You have no control
    - Dependent on a technology provider

# Serverless Technology for Mobile App Backends



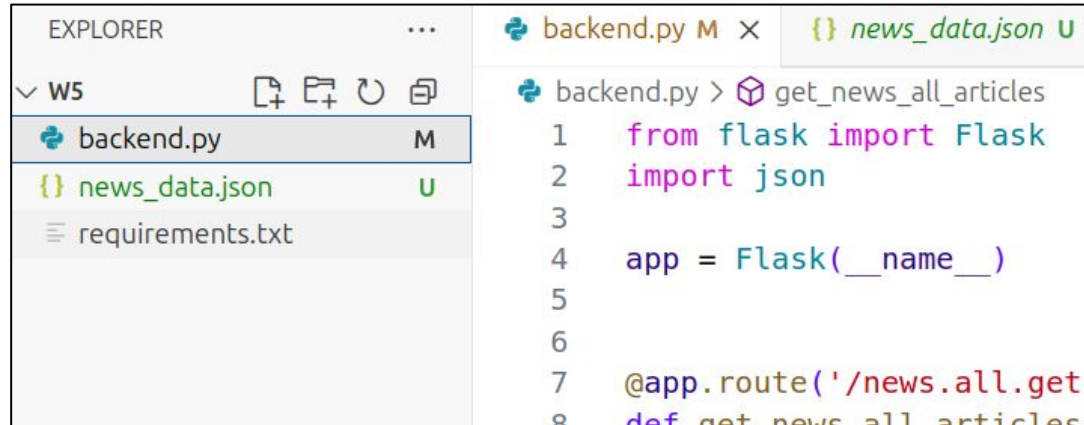
- **Serverless Technology Providers :**
  - **LeapCell.io**
  - **Render**
    - Supports even Laravel, Django/Python..
  - **AWS Lambda**
    - Very mature technology supporting most languages
  - **Netlify**
    - Supports JavaScript and TypeScript
  - **Firebase Functions**

# Serverless Technology for Mobile App Backends

- **Git Repository :**

- You may clone this repository:

- [https://github.com/imed5/mobile\\_backend\\_simple\\_flask/tree/master](https://github.com/imed5/mobile_backend_simple_flask/tree/master)



The screenshot shows a code editor interface. On the left, the 'EXPLORER' panel displays a file tree for a workspace named 'W5'. It contains three files: 'backend.py' (marked with a blue icon and 'M' for modified), 'news\_data.json' (marked with a green icon and 'U' for updated), and 'requirements.txt' (marked with a grey icon). On the right, the editor shows the content of 'backend.py'. The code is a Python Flask application snippet. It imports Flask and json, creates a Flask app, and defines a route for '/news.all.get' which calls a function 'get\_news\_all\_articles'.

```
EXPLORER
W5
  backend.py M
  news_data.json U
  requirements.txt

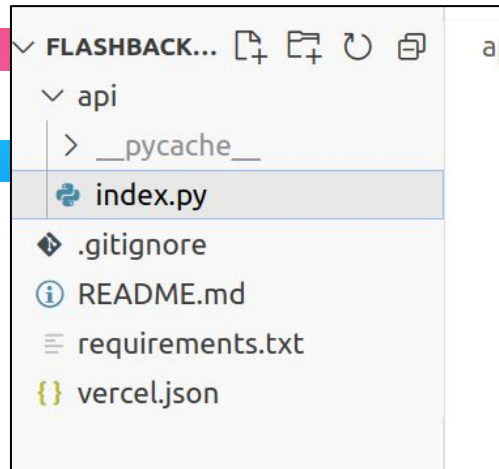
backend.py
1 from flask import Flask
2 import json
3
4 app = Flask(__name__)
5
6
7 @app.route('/news.all.get')
8 def get_news_all_articles
```

# Serverless Technology for Mobile App Backends

- **Git Repository : Project Structure**

- requirements.txt

```
Flask==3.0.0
gunicorn==21.2.0
```



# Serverless Technology for Mobile App Backends



- **Develop, Run & Test LOCALLY**

- To speed up
- To avoid paying for the deployment/building time

- Remember for Serverless, you may not need the main app.run method

```
if __name__ == "__main__":  
    app.run(port=8080)
```

- For local testing, use the command :

- **python3 backend** **OR**
- **flask --app backend run**



# Serverless Technology for Mobile App Backends



- **Get GIT Folder setup on your local machine:**
  - Create a folder
  - Inside the folder, execute the following commands
    - `git init`
    - `git remote add origin link_to_your_git_folder`  
`git remote add origin https://github.com/imed5/flask-product-info-no-database.git`
    - `git pull link_to_your_git_folder main`  
`git pull https://github.com/imed5/flask-product-info-no-database.git master`
    - `git branch --set-upstream-to=origin/master master`
    - `git pull`

# Serverless Technology for Mobile App Backends



- **Get GIT Folder setup on your local machine:**
  - `git checkout -b feature/YYYY_MM_DD/BRANCH_NAME`
  - `git add .`
  - `git commit -m "description of the changes"`
  - `git push origin feature/YYYY_MM_DD/BRANCH_NAME`
    - Make PR ( Pull Request )
    - Merge
  - `git checkout master`
  - `git pull`

# Serverless Technology for Mobile App Backends

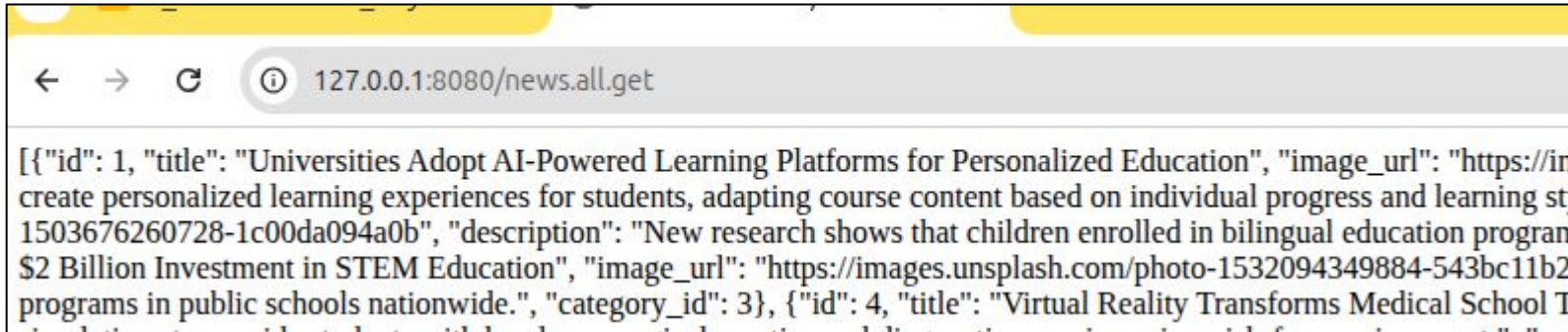
- Examples for getting cate

To view/Test the App, Open a browser on the address

<http://localhost:8080>

Or

<http://127.0.0.1:8080/news.all.get>



# Serverless Technology for Mobile App Backends

- Render.com & Git Integration

○

Search ^ K

**+ New**

Upgrade

?

**Build Command**  
Render runs this command to build your app before each deploy.

\$ pip install -r requirements.txt

**Start Command**  
Render runs this command to start your app with each deploy.

\$ gunicorn backend:app

Search ^ K

**+ New**

U

Static Site

**Web Service**

Private Service

Background Worker

Cron Job

Postgres

Key Value

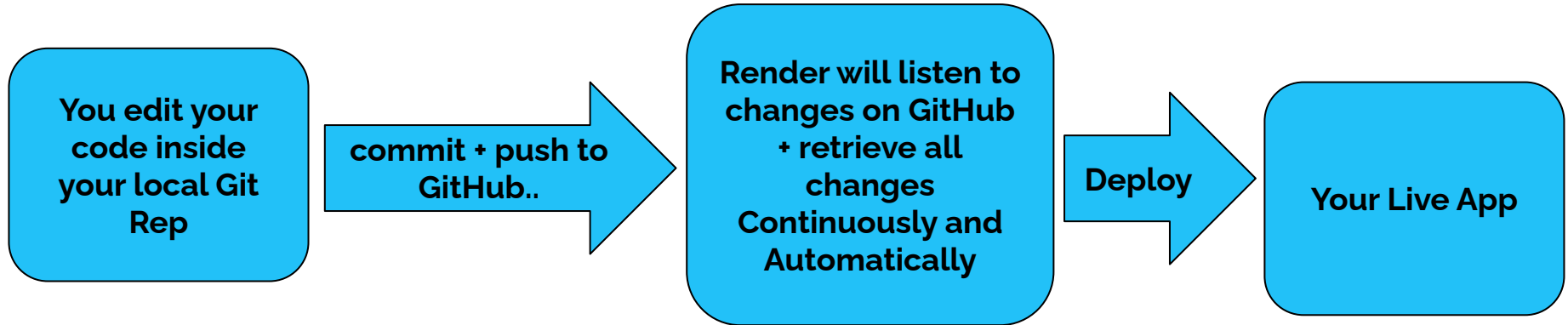
Project

Blueprint

PRO

# Serverless Technology for Mobile App Backends

- Continuous Integration/Continuous Delivery ( CI/CD ) :



# Deployment

## Runtime

Choose the runtime to run and deploy your service; the runtime includes pre-builts

Python



python3.11 debian

python3.11

debian

python3.11-slim-bookworm



<https://leapcell.io/>

## Build Command

The build command is used to build your codebase and is executed in the specified root directory, such as `npm install && npm run build` or `pip install -r requirements.txt`, for more examples, see [\[Examples\]](#)

```
pip install -r requirements.txt
```

## Start Command

The start command is used to start your service and is executed in the specified root directory, such as `npm run start` or `gunicorn -w 1 -b :8080 app:app`, for more examples, see [\[Examples\]](#)

```
gunicorn --workers 1 --bind :8080 --timeout 600 backend:app
```

## Serving Port

The port on which your service will operate, such as `8080`.

8080

Only Save

Save & Deploy

# Serverless Technology for Mobile App Backends

- Render.com & Git Integration

○

Search ^ K

**+ New**

Upgrade

?

**Build Command**  
Render runs this command to build your app before each deploy.

\$ pip install -r requirements.txt

**Start Command**  
Render runs this command to start your app with each deploy.

\$ gunicorn backend:app

Search ^ K

**+ New**

U

Static Site

**Web Service**

Private Service

Background Worker

Cron Job

Postgres

Key Value

Project

PRO

Blueprint

Dashboard

mobile\_backend\_simple\_flask

Events

Settings

MONITOR

Logs

Metrics

MANAGE

Environment

Shell

Scaling

Previews

Disks

Jobs

ⓘ Your free instance will spin down with inactivity, which can delay requests

All logs Search

```
Oct 21 11:32:27 AM ⓘ ==> Docs on specifying a port: https://re
Oct 21 11:36:44 AM ⓘ 2xj9c 127.0.0.1 - - [21/Oct/2025:10:36:44 +0000] "
7.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36"
Oct 21 11:52:46 AM ⓘ 2xj9c [2025-10-21 10:52:46 +0000] [56] [INFO] Hvice
Oct 21 11:52:46 AM ⓘ 2xj9c [2025-10-21 10:52:46 +0000] [57] [INFO] W
Oct 21 11:52:47 AM ⓘ 2xj9c [2025-10-21 10:52:47 +0000] [56] [INFO] S
d Worker
Oct 21 02:12:06 PM ⓘ ==> Deploying...
Oct 21 02:12:31 PM ⓘ r5h28 ==> Running 'gunicorn backend:app'
Oct 21 02:12:34 PM ⓘ r5h28 [2025-10-21 13:12:34 +0000] [56] [INFO] S
Oct 21 02:12:34 PM ⓘ r5h28 [2025-10-21 13:12:34 +0000] [56] [INFO] L
Oct 21 02:12:34 PM ⓘ r5h28 [2025-10-21 13:12:34 +0000] [56] [INFO] U
Oct 21 02:12:34 PM ⓘ r5h28 [2025-10-21 13:12:34 +0000] [57] [INFO] B
Oct 21 02:12:34 PM ⓘ r5h28 127.0.0.1 - - [21/Oct/2025:13:12:34 +0000]
Oct 21 02:12:37 PM ⓘ ==> Your service is live 🎉
Oct 21 02:12:37 PM ⓘ ==>
Oct 21 02:12:37 PM ⓘ ==> //////////////////////////////////////
Oct 21 02:12:37 PM ⓘ ==>
Oct 21 02:12:37 PM ⓘ ==> Available at your primary URL https://
Oct 21 02:12:37 PM ⓘ ==>
Oct 21 02:12:37 PM ⓘ ==> //////////////////////////////////////
```

+ New

⚡ U

Cre

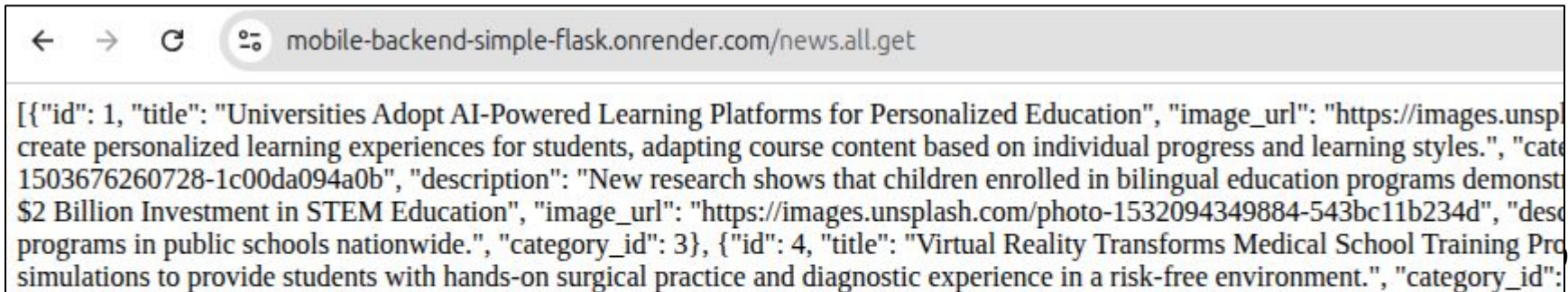
⚡ PRO



# Serverless Technology for Mobile App Backends

- Render.com & Git Integration

- <https://mobilebackendsimpleflask-imed1024-s73qk5q3.leapcell.dev/news.all.get>
- <https://mobile-backend-simple-flask.onrender.com/news.all.get>



# Section 3

**Async, Timer and FutureBuilder**





# Async Coding In Dart

# Writing Async Functions in Flutter



- **Synchronous Function/Instruction**

- Executed Sequentially, cannot move until the current instruction is fully completed.

- **Asynchronous Function/Instruction**

- Function or Set of instructions will be executed on a separate thread without blocking the main code.
  - The main program completes work while it may wait for “async” operations to finish in the future.

# Writing Async Functions in Flutter

- **Asynchronous Function :**

- Common asynchronous operations include :
  - Fetching/Sending data over a network.
  - Reading/Writing to a database.
  - Writing/Reading data from a local file.
  - Opening/Communicating with the device hardware
  - ..

# Writing Async Functions in Flutter



- **Async**

- To declare a function as asynchronous, use the **async** keyword before the body.

```
void getData() async{  
    //some business logic here..  
}
```

# Writing Async Functions in Flutter



- **Return Type of Asynchronous Functions :**

- Asynchronous functions in Dart/Flutter **return** Future Objects.
- For Async Functions, the return **must** be always a future. (or other types of similar nature)
  - `String → Future <String>`
  - `int → Future<int>`
  - `Map → Future<Map>`
  - `void → Future<void>`

# Writing Async Functions in Flutter

- **Return Type of Asynchronous Functions :**

- Asynchronous functions in Dart/Flutter **return** Future Objects.
- For Async Functions, the return **must** be always a future. (or other types of similar nature)

- String → Future <String>
- int → Future<int>
- Map → Future<Map>
- **void → Future<void> [ Avoid to use it]**

```
Future<String> myStr=getData() ;  
Future<Map> mydata=getData() ;
```



# Writing Async Functions in Flutter

- **Await**

- The **await** will make an asynchronous instruction as a synchronous one
  - Wait until the instruction completes and proceeds to the next instruction

```
some _ instruction  
myData = await getData() ;  
some _ instruction
```

# Writing Async Functions in Flutter

- **Await**

- The **await** will make a  
  - Wait until the instruction  
instruction

```
some _ instruction  
myData = await getData() ;  
some _ instruction
```

What type of myData ?  
Future or non-Future

# Writing Async Functions in

**Very IMPORTANT**

- **A** IF YOU USE `await`, the async FUNCTION MUST

**RETURN A NON-VOID VALUE**

`void insertData(..) async {` ⇒ NO

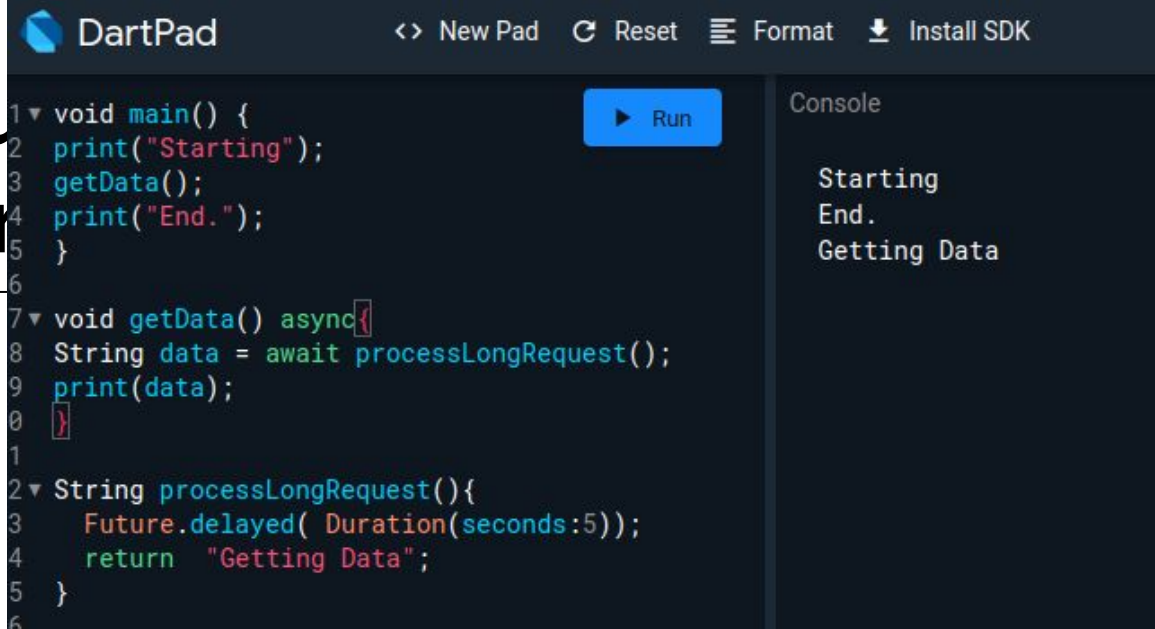
`Future<bool> insertData(..) async {` ⇒ Yes

# Writing Async Functions in Flutter

```
void main() {  
    print("Starting");  
    getData();  
    print("End.");  
}  
  
void getData() async{  
    String data = await processLongRequest();  
    print(data);  
}  
  
String processLongRequest() {  
    Future.delayed( Duration(seconds:5) );  
    return "Getting Data" ;  
}
```

# Writing Async Fu Flutter

```
void main() {  
    print("Starting");  
    getData();  
    print("End.");  
}  
  
void getData() async{  
    String data = await processLongRequest();  
    print(data);  
}  
  
String processLongRequest() {  
    Future.delayed( Duration(seconds:5));  
    return "Getting Data" ;  
}
```



DartPad

<> New Pad    ↺ Reset    ≡ Format    ⬇ Install SDK

Run

```
1 void main() {  
2   print("Starting");  
3   getData();  
4   print("End.");  
5 }  
6  
7 void getData() async{  
8   String data = await processLongRequest();  
9   print(data);  
10 }  
11  
12 String processLongRequest(){  
13   Future.delayed( Duration(seconds:5));  
14   return "Getting Data";  
15 }  
16
```

Console

Starting  
End.  
Getting Data

**Incorrect Code**

# Writing Async Functions in Flutter

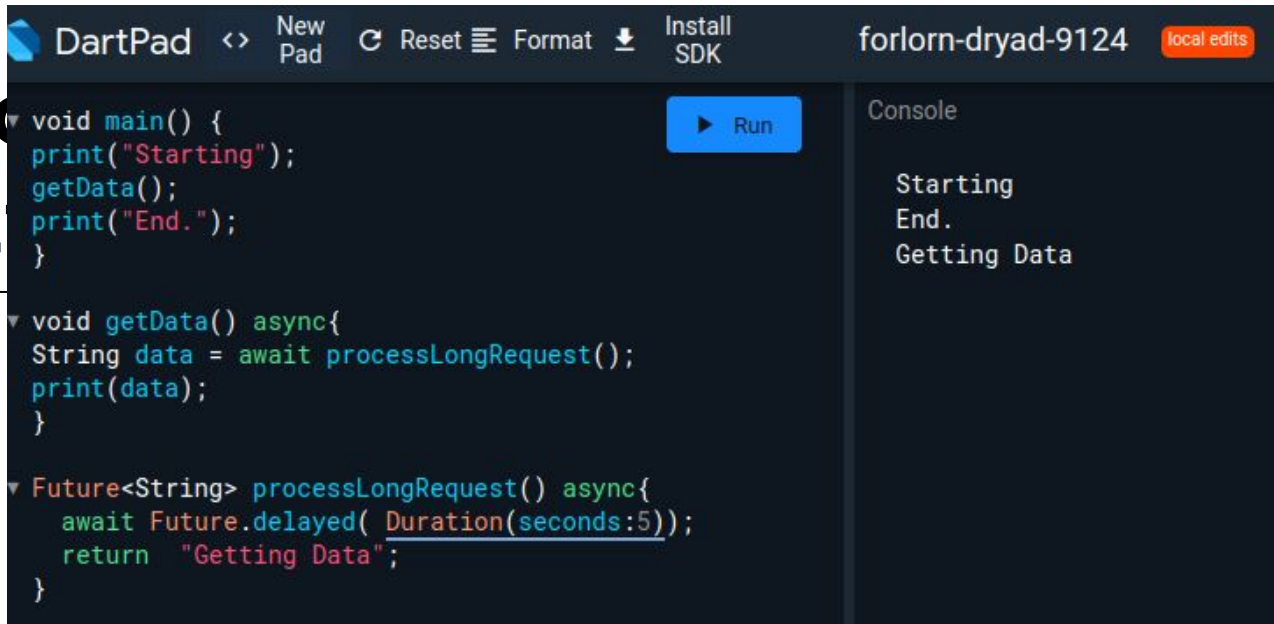
```
void main() {  
    print("Starting");  
    getData();  
    print("End.");  
}  
  
void getData() async{  
    String data = await processLongRequest();  
    print(data);  
}  
  
Future<String> processLongRequest() async{  
    await Future.delayed( Duration(seconds:5));  
    return "Getting Data" ;  
}
```

# Writing Async Fluently

```
void main() {  
    print("Starting");  
    getData();  
    print("End.");  
}
```

```
void getData() async{  
    String data = await processLongRequest();  
    print(data);  
}
```

```
Future<String> processLongRequest() async{  
    await Future.delayed( Duration(seconds:5));  
    return "Getting Data" ;  
}
```



The screenshot shows the DartPad web editor interface. The top bar includes the DartPad logo, a code icon, a 'New Pad' button, and buttons for 'Reset', 'Format', and 'Install SDK'. The username 'forlorn-dryad-9124' and a 'local edits' indicator are on the right. The code editor contains the following Dart code:

```
void main() {  
    print("Starting");  
    getData();  
    print("End.");  
}  
  
void getData() async{  
    String data = await processLongRequest();  
    print(data);  
}  
  
Future<String> processLongRequest() async{  
    await Future.delayed( Duration(seconds:5));  
    return "Getting Data";  
}
```

A 'Run' button is located to the right of the code editor. The console output on the right side of the editor shows the execution results:

```
Starting  
End.  
Getting Data
```

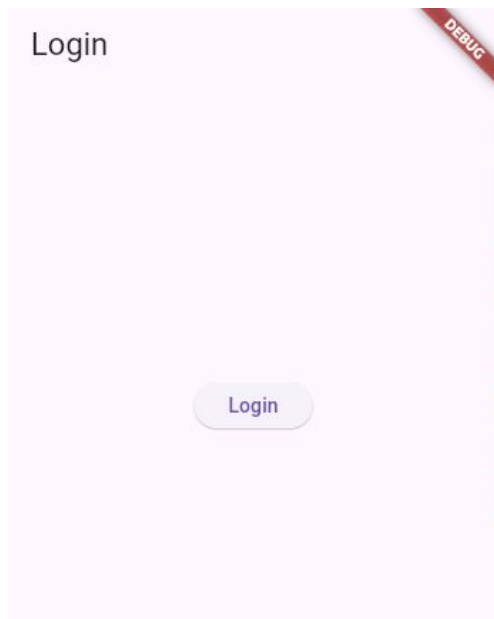


# Future In Flutter

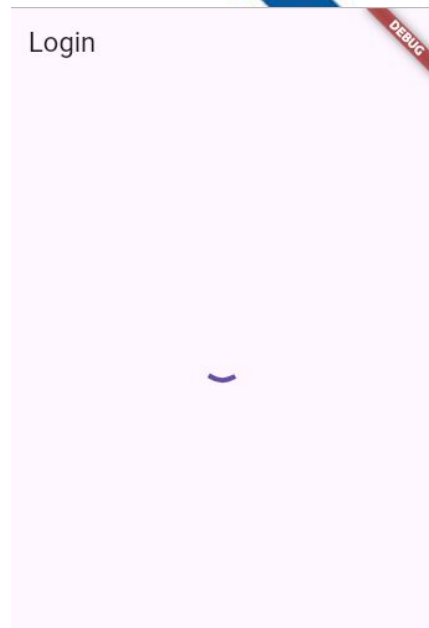


# Using Future Functions in Flutter

- Case Study/ Example



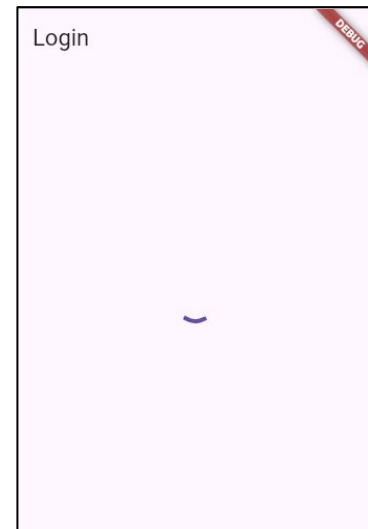
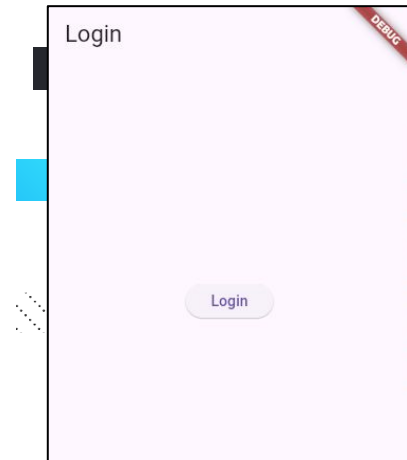
**While waiting for the Future  
Function  
Login form is hidden  
Circular Progress Bar is shown..**



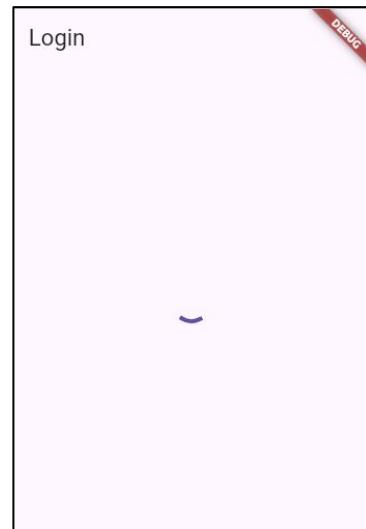
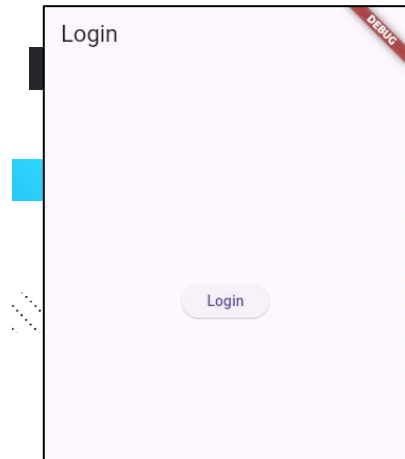
```

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});
  @override
  State<LoginScreen> createState () => _LoginScreenState ();
}
class _LoginScreenState extends State<LoginScreen> {
  bool is loading = false;
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Login')),
      body: Container(
        margin: EdgeInsets.all(16),
        child: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Visibility(
                visible: !is loading,
                child: ElevatedButton(
                  onPressed: () {
                    processLogin();
                  },
                  child: Text('Login'),
                ),
              ),
              Visibility(
                visible: is loading,
                child: CircularProgressIndicator(),
              ),
            ],
          ),
        ),
      ),
    ),
  ),
),

```



```
Future<bool> processLogin() async {  
  setState(() {  
    is_loading = true;  
  });  
  
  await Future.delayed(Duration(seconds: 5));  
  
  setState(() {  
    is_loading = false;  
  });  
  
  ScaffoldMessenger.of(  
    context,  
  ).showSnackBar(SnackBar(content: Text('Login Successful')));  
  
  return true;  
}  
}
```



# FutureBuilder Widget in Flutter



- **FutureBuilder**

- It is a special widget for **building UI** based on data assigned to a Future Object.
- The widget will monitor the future object, once it has a snapshot of data ready, it will build the widget.

# FutureBuilder Widget in Flutter



- **FutureBuilder**

- The main attributes of the future builder :
  - **future** : represents the Future variable whose snapshot can be accessed by the builder function.
  - **builder** : This property represents the current build strategy.
  - **initialData** : represents the data that will be utilized to build the snapshots until a non-null Future has been completed.

# FutureBuilder Widget in Flutter

- FutureBuilder

- Very simple example

```
Future<String> getCurrentTime() async {  
    await Future.delayed(Duration(seconds: 5)); // Simulation only  
    DateTime time = DateTime.now();  
    String strTime =  
        "${time.year}-${time.month}-${time.day} ${time.hour}:${time.minute}";  
    return strTime;  
}
```

```

import 'package:flutter/material.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  Future<String> getCurrentTime() async {
    await Future.delayed(Duration(seconds: 5));
    DateTime time = DateTime.now();
    String strTime =
      "${time.year}-${time.month}-${time.day} ${time.hour}:${time.minute}";
    return strTime;
  }

  @override
  Widget build(BuildContext context) {
    Future<String> strTime = getCurrentTime();

    return Scaffold(
      appBar: AppBar(title: Text("Current Time")),
      body: Center(
        child: Column(
          children: [
            SizedBox(height: 20),
            Text(strTime.toString()),
          ],
        ),
      ),
    );
  }
}

```

## Current Time

DEBUG

Instance of 'Future<String>'

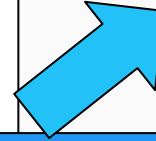
...

```
class _HomeScreenState extends State<HomeScreen> {  
  Future<String> getCurrentTime() async {  
    await Future.delayed(Duration(seconds: 5));  
    DateTime time = DateTime.now();  
    String strTime =  
      "${time.year}-${time.month}-${time.day} ${time.hour}:${time.minute}";  
    return strTime;  
  }  
  @override  
  Widget build(BuildContext context) {  
    Future<String> strTime = getCurrentTime();  
    return Scaffold(  
      appBar: AppBar(title: Text("Current Time")),  
      body: Center(  
        child: Column(  
          children: [  
            SizedBox(height: 20),  
            FutureBuilder<String>(  
              future: strTime,  
              builder: (context, snapshot) {  
                if (snapshot.hasData) {  
                  return Text(snapshot.data!);  
                } else if (snapshot.hasError) {  
                  return Text("${snapshot.error}");  
                }  
                return CircularProgressIndicator();  
              })  
          ],  
        ),  
      ),  
    );  
  }  
}
```

Current Time

DEBUG

2023-11-28 16:22



Current Time

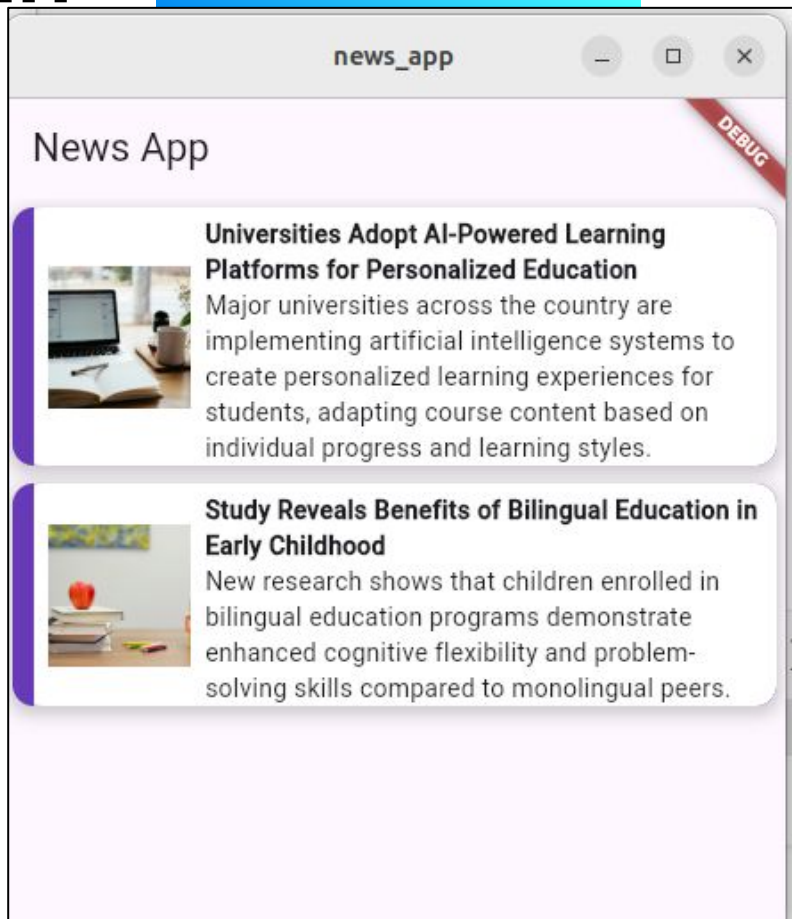
DEBUG





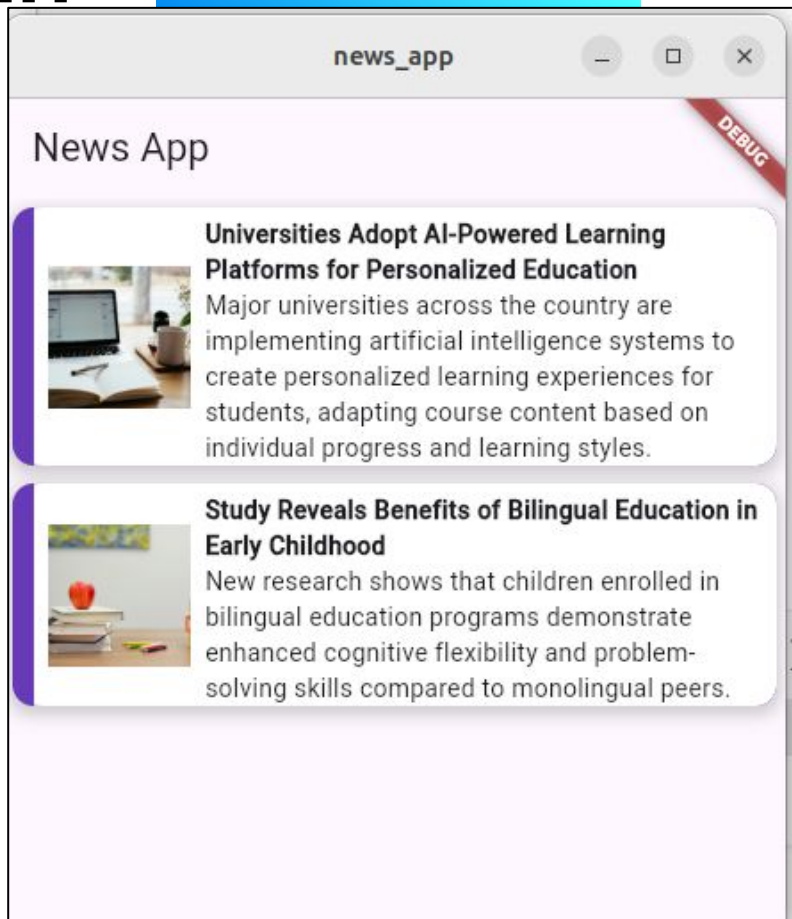
# FutureBuilder Widget in Flutter

- Example of FutureBuilder using Map
  - What's the Widget Tree ?



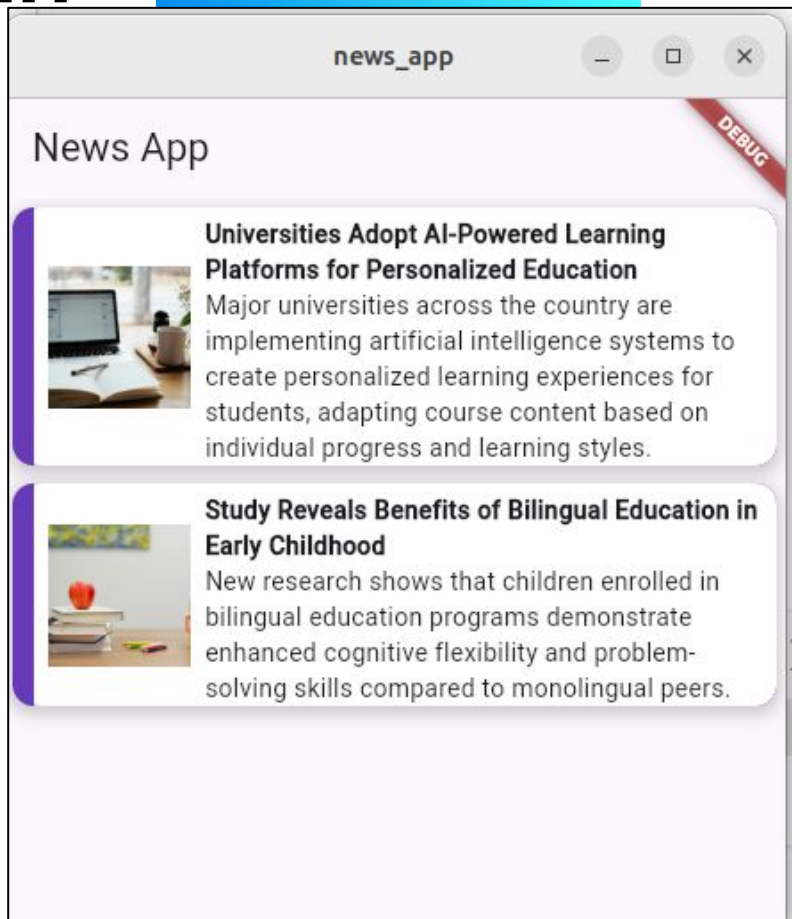
# FutureBuilder Widget in Flutter

- **Example of FutureBuilder using Map**
  - What's the Widget Tree ?
    - Center:
      - ListView.Builder
        - **NewsItem Widget**



# FutureBuilder Widget in Flutter

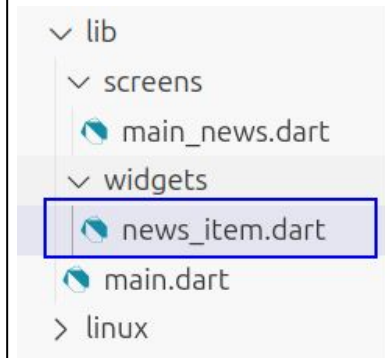
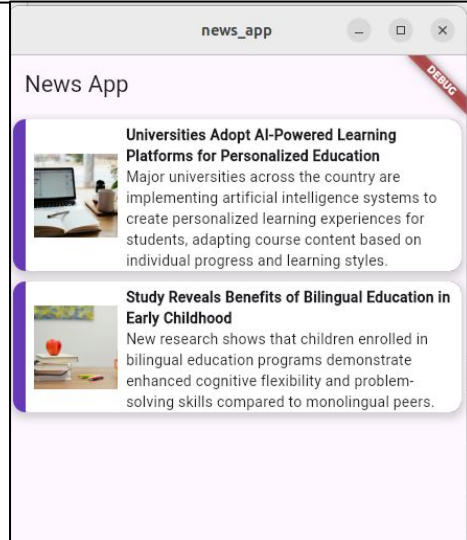
- Example of FutureBuilder using Map
  - What's the Widget Tree ?
    - **NewsItem Widget**
      - **Container**
        - **Row**
          - **Image.Network**
          - **Column**
            - **Text + Text**



```

class NewsItem extends StatelessWidget {
  final Map data;
  const NewsItem({super.key, required this.data});
  @override
  Widget build(BuildContext context) {
    return Container (
      constraints: BoxConstraints (
        minHeight: 100, // minimum height
      ),
      margin: EdgeInsets.all(5),
      decoration: BoxDecoration ( ... ),
      child: Row (
        children: [
          SizedBox (width: 8),
          Image.network (
            data['image_url'], width: 80,height: 80,fit: BoxFit.cover),
          SizedBox (width: 8),
          Expanded (
            child: Column (
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                SizedBox (height: 5),
                Text (
                  data['title'],
                  style: TextStyle (fontWeight: FontWeight.bold),
                ),
                Text (data['description']),
              ],
            ),
          ),
          SizedBox (width: 8),
        ],
      ),
    );
  }
}

```

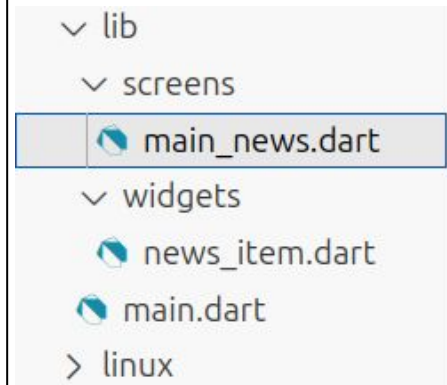
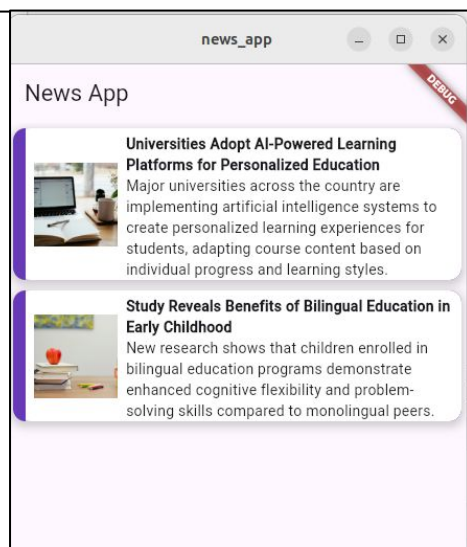


```

class NewsScreen extends StatefulWidget {
  const NewsScreen({super.key});
  @override
  State<NewsScreen> createState() => _NewsScreenState();
}

class _NewsScreenState extends State<NewsScreen> {
  List<Map> news = [
    {
      "id": 1,
      "Title": "Universities ...",
      "Image_url": "https://images.unsplash.com/photo-1501504905252-473c47e087f8",
      "Description": "Major universities across...",
      "category_id": 3,
    },
  ];
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('News App')),
      body: Center(
        child: ListView.builder(
          itemCount: news.length,
          itemBuilder: (context, index) {
            return NewsItem(data: news[index]);
          },
        ),
      ),
    );
  }
}

```



```

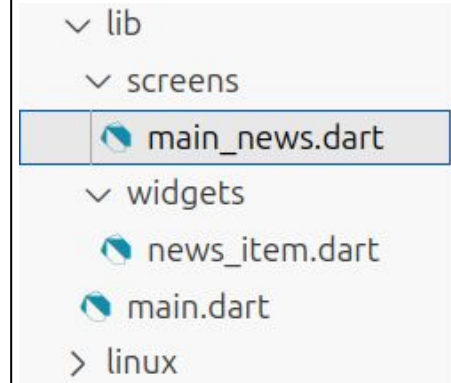
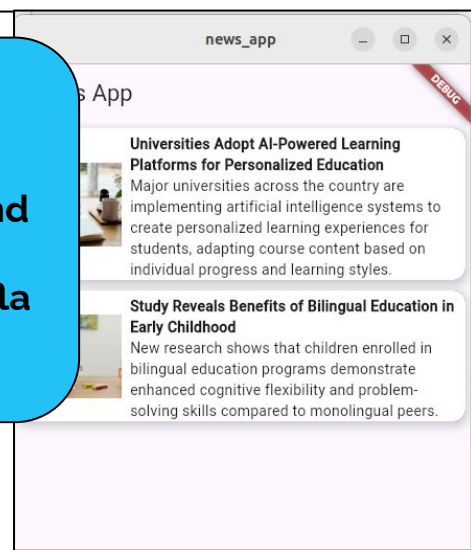
class NewsScreen extends StatefulWidget {
  const NewsScreen({super.key});
  @override
  State<NewsScreen> createState() => _NewsScreenState;
}
class _NewsScreenState extends State<NewsScreen> {
  List<Map> news = [
    {
      "id": 1,
      "Title": "Universities ...",
      "Image_url": "https://images.unsplash.com...",
      "Description": "Major universities across...",
      "category_id": 3,
    },
  ];
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('News App')),
      body: Center(
        child: ListView.builder(
          itemCount: news.length,
          itemBuilder: (context, index) {
            return NewsItem(data: news[index]);
          },
        ),
      ),
    );
  }
}

```

**Data is hardcoded ?**

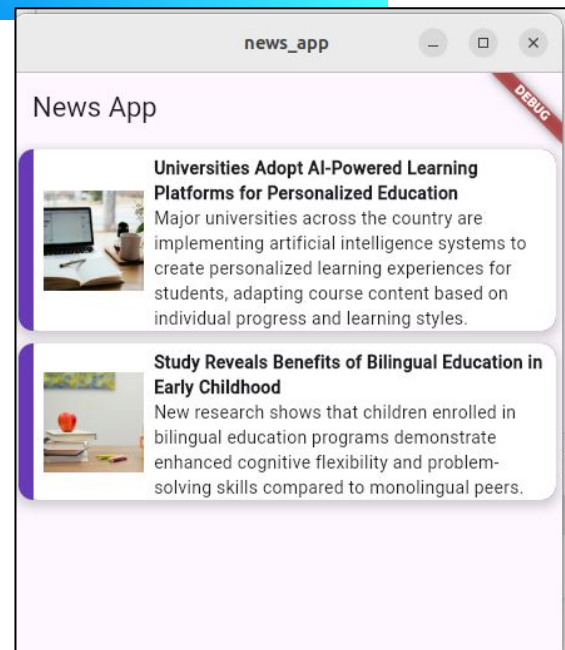
**How to load data from the Backend  
Endpoint ?**

**<https://mobile-backend-simple-flask.onrender.com/news.all.get>**



# FutureBuilder Widget in Flutter

- **Simple Steps to load news data from Web**
  - Do I need to load the news:
    1. Every time I refresh Widget UI?
    2. When the widget is created the first time ?



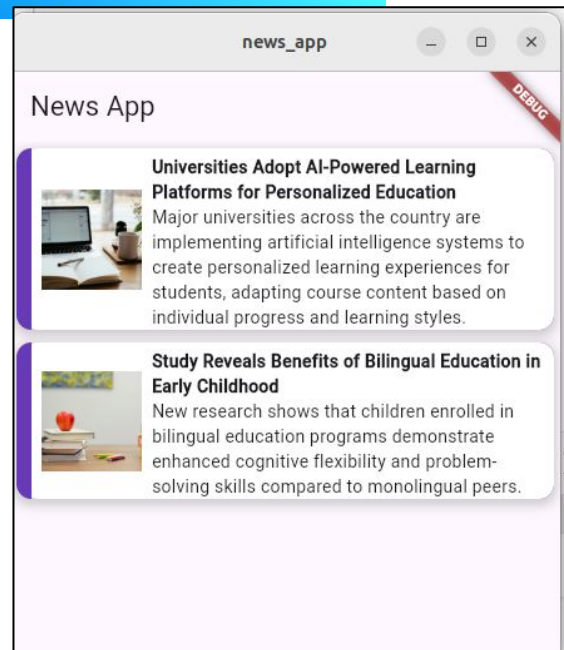
# FutureBuilder Widget in Flutter

- Simple Steps to load news data from Web

1. Every time I refresh Widget UI?

1. Optionally, initialize the Future Variable

```
class _NewsScreenState extends State<NewsScreen> {  
  late Future<List<Map>>> news;  
  
  @override  
  Widget build(BuildContext context) {  
    news = NewsUtils.getNewsFromWeb();  
  }  
}
```





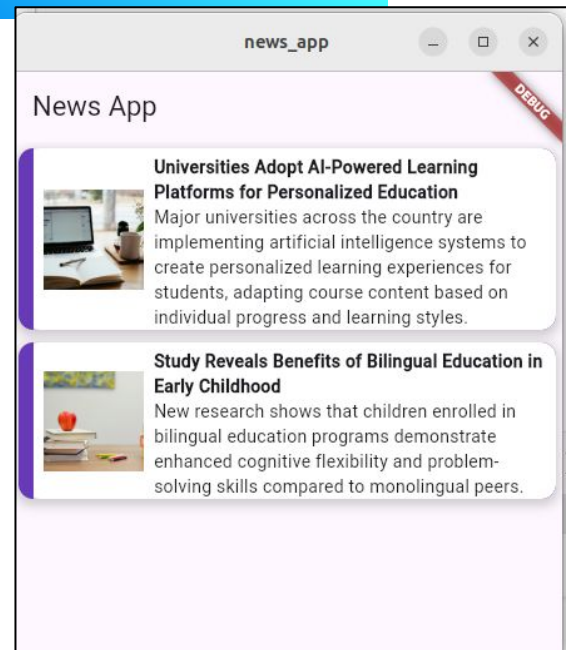
# FutureBuilder Widget in Flutter

- Simple Steps to load news data from Web

1. Every time I refresh Widget UI?

2. Use the FutureBuilder :

- Set the future attribute
- Builder Function States when :
  - There is an error
  - Data received
  - Still waiting ..



# FutureBuilder Widget in Flutter

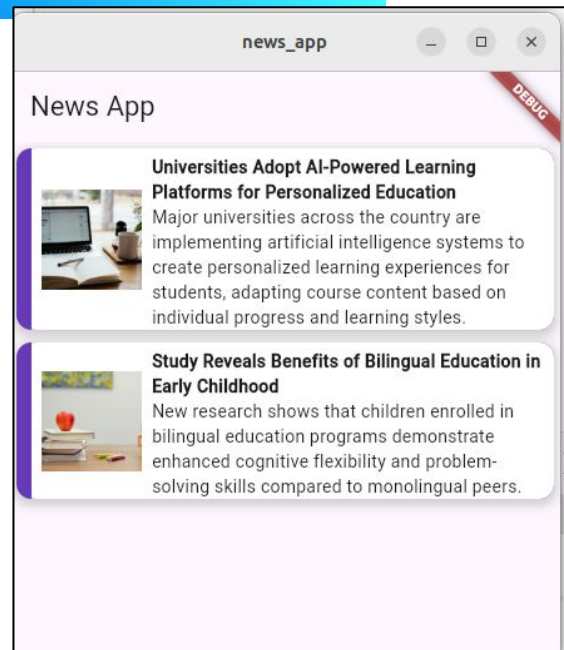
```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: const Text('News App')),  
    body: Center(  
      child: ListView.builder(  
        itemCount: news.length,  
        itemBuilder: (context, index) {  
          return NewsItem(data: news[index]);  
        },  
      ),  
    ),  
  );  
}
```



```
Widget build(BuildContext context) {  
  news = NewsUtils.getNewsFromWeb();  
  return Scaffold(  
    appBar: AppBar(title: const Text('News App')),  
    body: Center(  
      child: FutureBuilder<List<Map>>(  
        future: news,  
        builder: (context, snapshot) {  
          if (snapshot.hasError) return Text('Error: ${snapshot.error}');  
  
          if (snapshot.hasData) {  
            List<Map> fetched_news = snapshot.data ?? [];  
            return ListView.builder(  
              itemCount: fetched_news.length,  
              itemBuilder: (context, index) {  
                return NewsItem(data: fetched_news[index]);  
              },  
            );  
          }  
          return CircularProgressIndicator();  
        },  
      ),  
    ),  
  );  
}
```

# FutureBuilder Widget in Flutter

- **Simple Steps to load news data from Web**
  - Do I need to load the news:
    1. Every time I refresh Widget UI?
    2. **When the widget is created the first time ?**



# FutureBuilder Widget in

- Simple S

```
Widget build(BuildContext context) {  
  news = NewsUtils.getNewsFromWeb();  
  return Scaffold(  
    appBar: AppBar(title: const Text('News App')),  
    body: Center(  
      child: FutureBuilder<List<Map>>(  

```

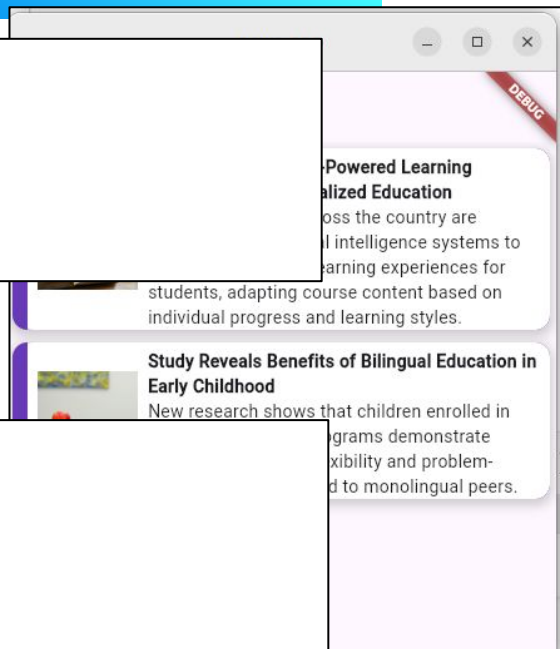
- Do I need to load the news:

1. Every time I refresh Widget UI?

- 2.

```
@override  
void initState() {  
  super.initState();  
  news = NewsUtils.getNewsFromWeb();  
}  
  
@override  
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: const Text('News App')),  

```



# FutureBuilder Widget in Flutter

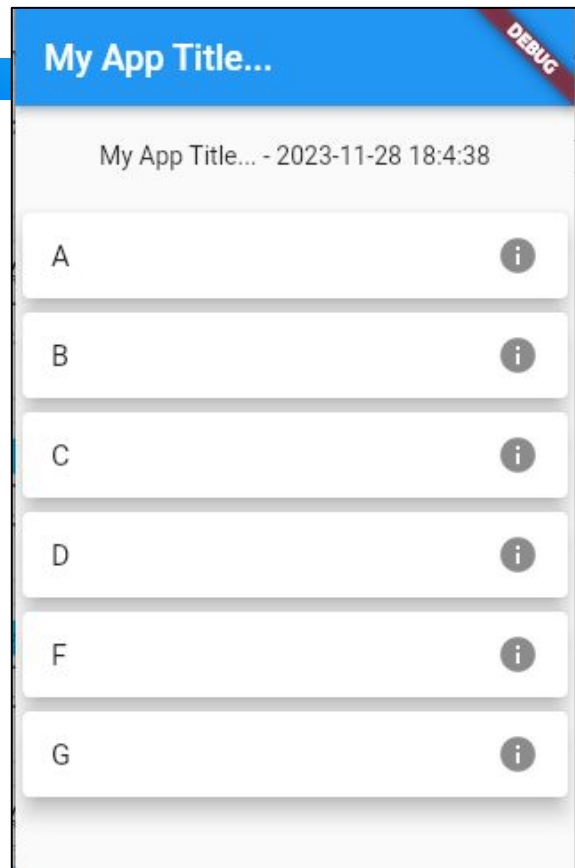
- **Slide to Refresh the time and data**

- Widget to use :

- **RefreshIndicator**

- child: widget
      - onRefresh: function

**Widget Available only for Mobile  
Devices ( Not web or desktop app)**



```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text(appBarTitle)),
    body: Center(
      child: FutureBuilder<Map<String, dynamic>>(
        future: myInfo,
        builder: (context, snapshot) {
          if (snapshot.hasData) {
            Map myData = snapshot.data!;
            return RefreshIndicator(
              child: getInformationWidget(myData),
              onRefresh: _refreshMyData
            );
          } else if (snapshot.hasError) {
            return Text("${snapshot.error}");
          }

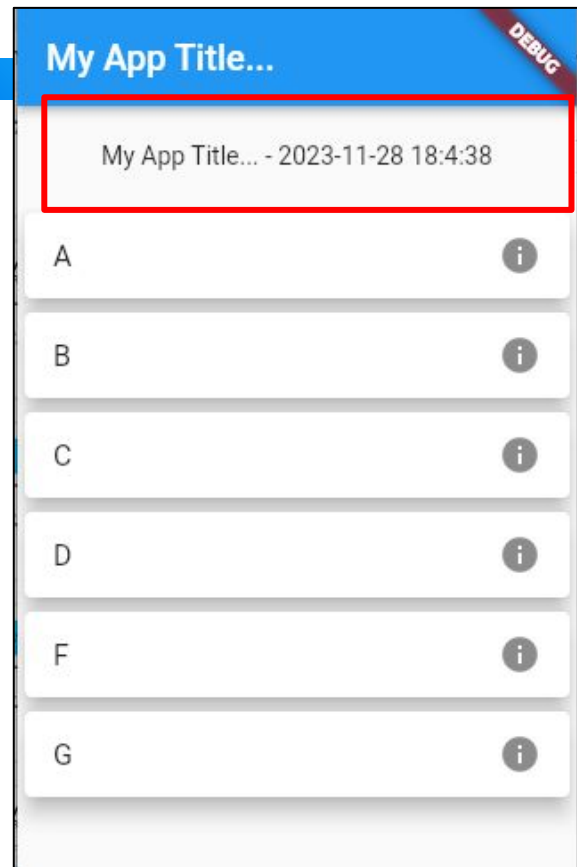
          return CircularProgressIndicator();
        }
      )
    )
  );
}

```

```

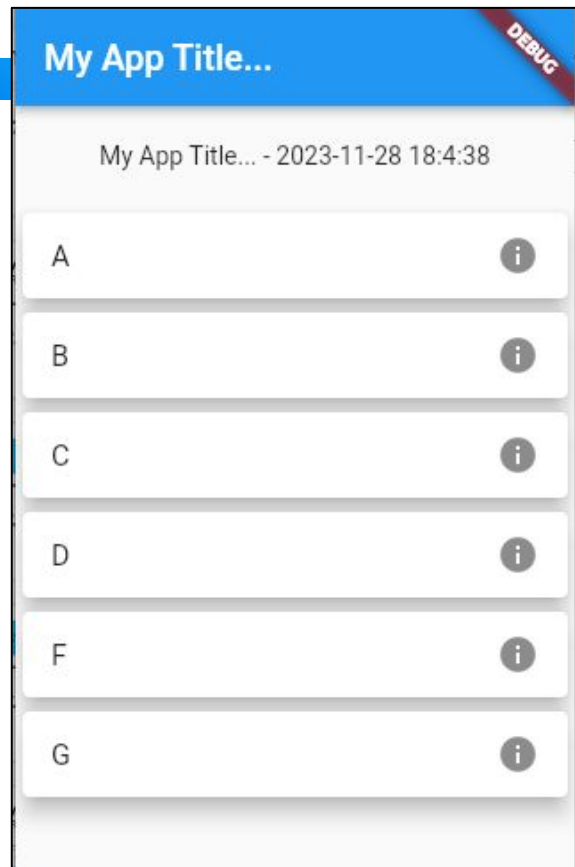
Future<void> _refreshMyData() async {
  myInfo = getComplexData();
  setState(() {});
  return;
}

```



# Writing Async Functions in Flutter

- **Automated Update of the Data every 1 minute :**
  - What library to use ?



```
import 'dart:async';
```

```
...
```

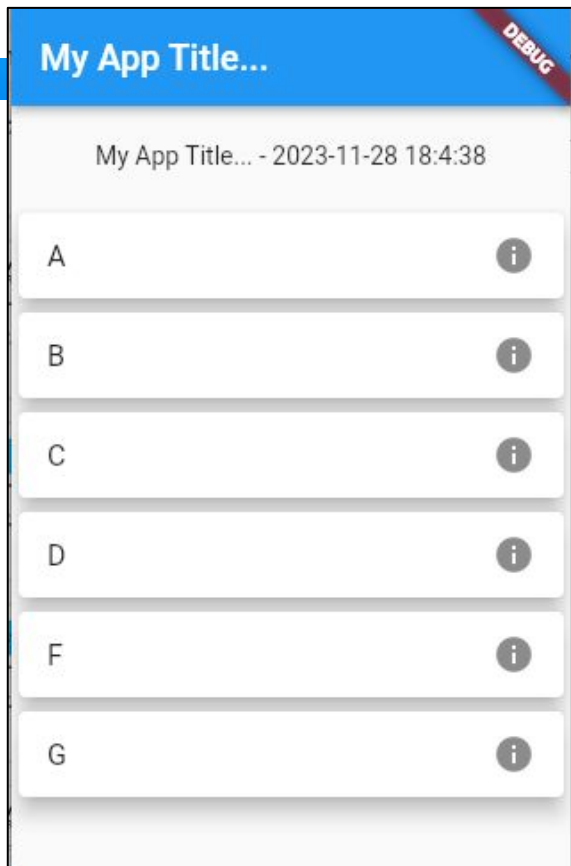
```
@override
```

```
void initState() {  
  super.initState();
```

```
    Timer.periodic(Duration(seconds: 30), (Timer t) {  
      print("Getting Data and Update the UI");  
      _refreshMyData();  
    });
```

```
    myInfo = getComplexData();  
  }
```

```
Future<void> _refreshMyData() async {  
  myInfo = getComplexData();  
  setState(() {});  
  return;  
}
```





```
import 'dart:async';
```

```
...
```

```
@override
```

```
void initState() {  
  super.initState();
```

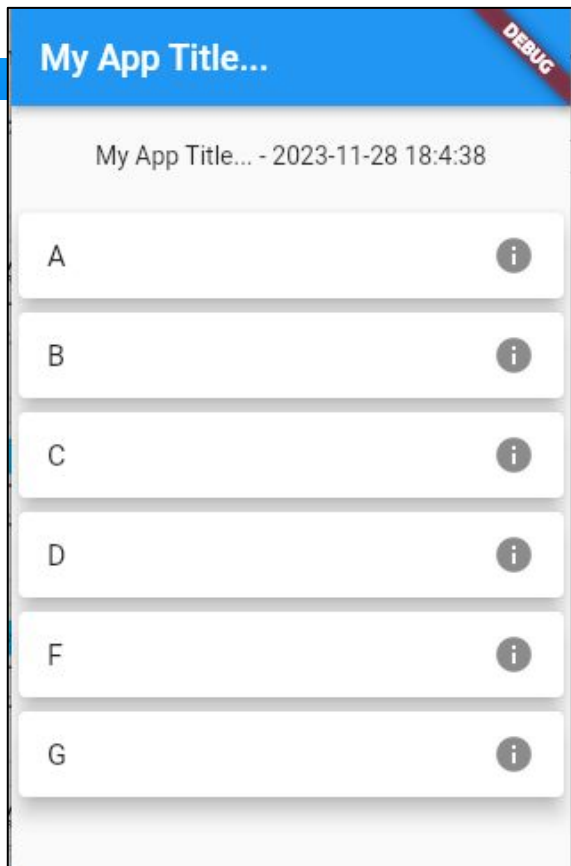
```
  Timer.periodic(Duration(seconds: 30), (Timer t) {  
    print("Getting Data and Update the UI");  
    _refreshMyData();  
  });
```

```
  myInfo = getComplexData();  
}
```

```
Future<void> _refreshMyData() async {  
  myInfo = getComplexData();  
  setState(() {});  
  return;  
}
```

**getComplexData is Async ?**

**setState( () {} ) is it redundant ?**



```
import 'dart:async';
```

```
...
```

```
@override
```

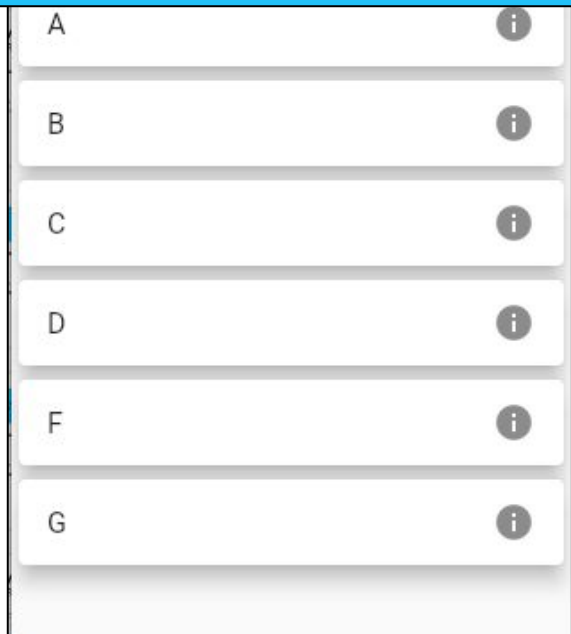
```
void initState() {  
  super.initState();
```

```
  Timer.periodic(Duration(seconds: 30), (Timer t) {  
    print("Getting Data and Update the UI");  
    _refreshMyData();  
  });
```

```
  myInfo = getComplexData();  
}
```

```
Future<void> _refreshMyData() async {  
  myInfo = getComplexData();  
  setState(() {});  
  return;  
}
```

**The use of setState is totally misleading here as it does nothing or it will show inconsistent + outdated data.**



```
import 'dart:async';

...

@override
void initState() {
  super.initState();

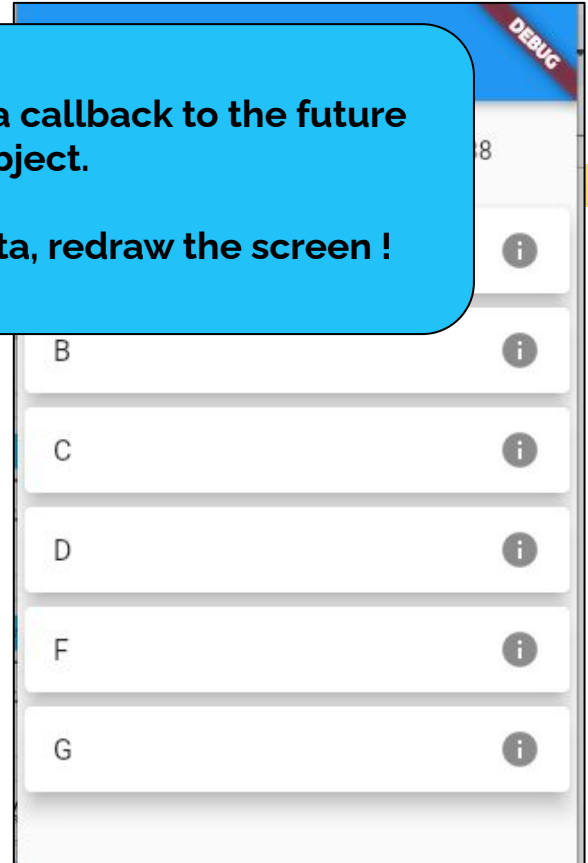
  Timer.periodic(Duration(seconds: 30), (Time
    print("Getting Data and Update the UI");
    _refreshMyData();
  });

  myInfo = getComplexData();
}

Future<void> _refreshMyData() async {
  myInfo = getComplexData();
  myInfo.then((_) => setState(() {}));
  return;
}
```

**We are registering a callback to the future object.**

**When you have data, redraw the screen !**



```
import 'dart:async';
```

```
...
```

```
@override
```

```
void initState() {  
  super.initState();
```

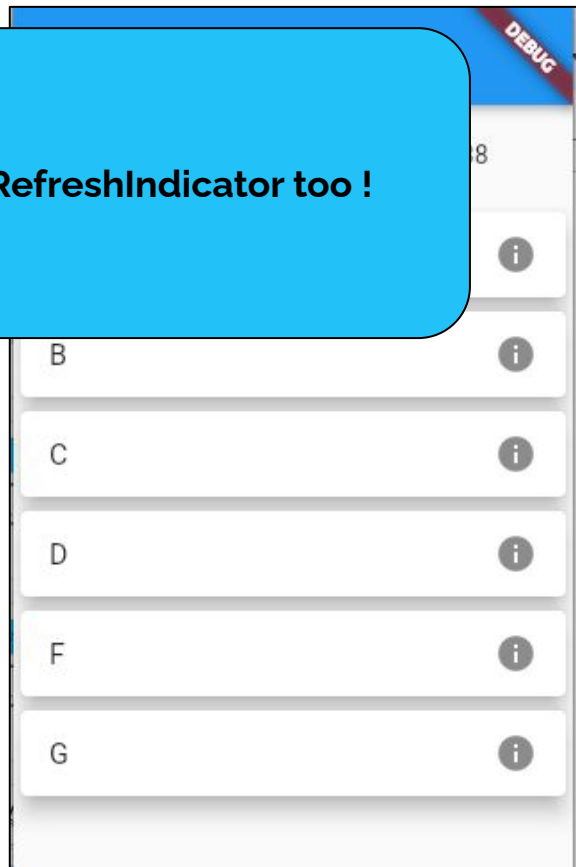
```
    Timer.periodic(Duration(seconds: 30), (Time  
      print("Getting Data and Update the UI");  
      _refreshMyData();  
    });
```

```
    myInfo = getComplexData();
```

```
}
```

```
Future<void> _refreshMyData() async {  
  myInfo = getComplexData();  
  myInfo.then((_) => setState(() {}));  
  return;  
}
```

**The same for the RefreshIndicator too !**





# Use of Design Patterns

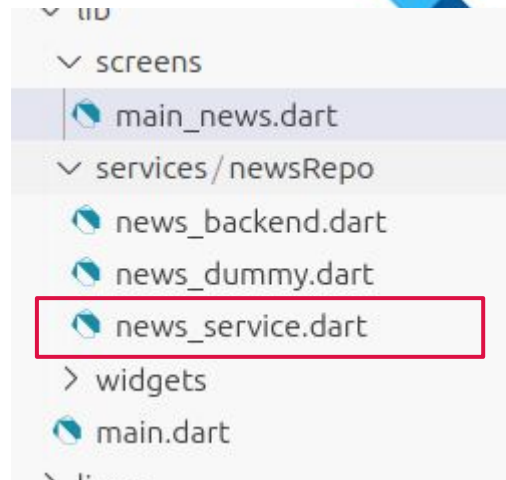
# Abstraction & Design Pattern for Structuring

```
import
'package:news_app/services/newsRepo/news_dummy.dart';

abstract class AbstractNewsRepo {
  Future<List<Map>> getNewsData();

  static AbstractNewsRepo? _newsInstance;

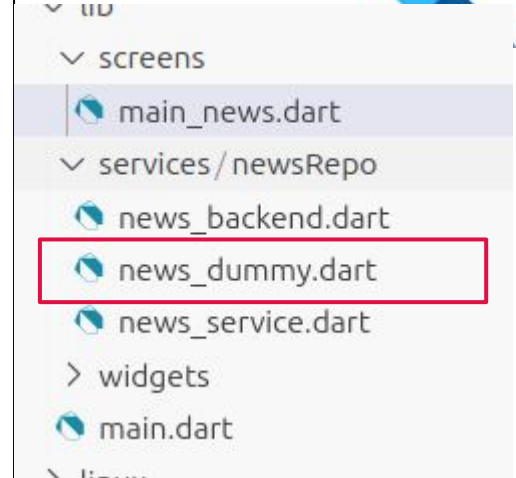
  static AbstractNewsRepo getInstance() {
    _newsInstance ??= NewsDummy();
    return _newsInstance!; // For backend data
  }
}
```



```

class NewsDummy extends AbstractNewsRepo {
  @override
  Future<List<Map>> getNewsData() async {
    await Future.delayed(Duration(seconds: 5)); // Simulate network delay
    List<Map> news = [
      {
        "id": 1,
        "title":
          "Universities Adopt AI-Powered Learning Platforms for Personalized
Education",
        "image_url":
          "https://images.unsplash.com/photo-1501504905252-473c47e087f8" ,
        "description":
          "Major universities across the country are implementing artificial
intelligence systems to create personalized learning experiences for students,
adapting course content based on individual progress and learning styles." ,
        "category_id": 3,
      },
      {
        "id": 2,
        "title":
          "Study Reveals Benefits of Bilingual Education in Early Childhood" ,
        "image url":
          "https://images.unsplash.com/photo-1503676260728-1c00da094a0b" ,
        "description":
          "New research shows that children enrolled in bilingual education
programs demonstrate enhanced cognitive flexibility and problem-solving skills
compared to monolingual peers." ,
        "category_id": 3,
      },
    ];
    return news;
  }
}

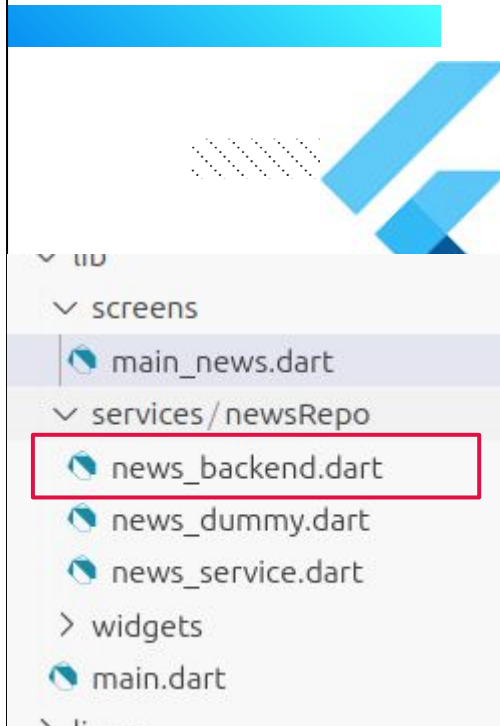
```



## flutter pub add http

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:news_app/services/newsRepo/news_service.dart';

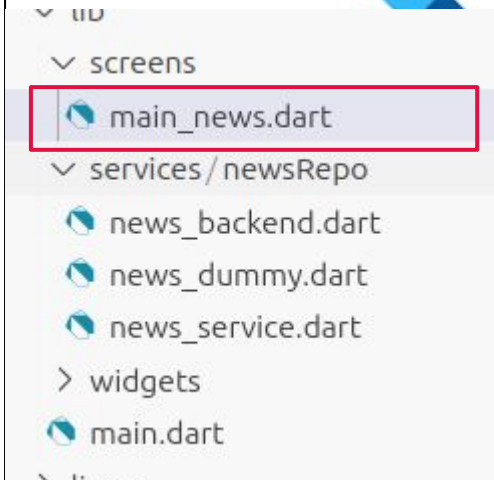
class NewsBackend extends AbstractNewsRepo {
  @override
  Future<List<Map>> getNewsData() async {
    print('Function : getNewsFromWeb : Fetching news from web...');
    try {
      final response = await http.get(
        Uri.parse(
          'https://mobilebackendsimpleflask-imed1024-s73qk5q3.leapcell.dev/news.al
1.get ',
        ),
        headers: {'Content-Type': 'application/json'},
      );
      print('Response status: ${response.toString()}');
      if (response.statusCode == 200) {
        // Parse JSON response
        final List<dynamic> data = json.decode(response.body);
        return data.map((item) => item as Map<String, dynamic>).toList();
      } else {
        throw Exception('Failed to load news: ${response.statusCode}');
      }
    } catch (e) {
      throw Exception('Error fetching news: $e');
    }
  }
}
```





# Abstraction & Design Pattern for Structuring

```
class _NewsScreenState extends State<NewsScreen> {  
  late Future<List<Map>>> news;  
  late AbstractNewsRepo newsRepo;  
  @override  
  void initState() {  
    super.initState();  
    newsRepo = AbstractNewsRepo.getInstance();  
    news = newsRepo.getNewsData();  
  }  
}
```

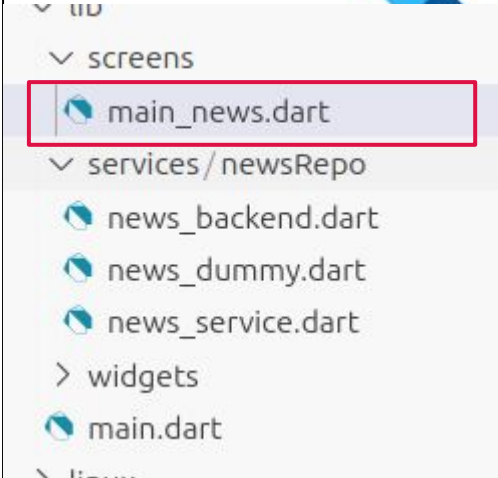


# Abstraction & Design Pattern for Structuring

```
class _NewsScreenState extends State<NewsScreen> {  
  late Future<List<Map>> news;  
  late AbstractNewsRepo newsRepo;  
  @override  
  void initState() {  
    super.initState();  
    newsRepo = AbstractNewsRepo.getInstance();  
    news = newsRepo.getNewsData();  
  }  
}
```

Can we just simply say :

```
newsRepo = new BackendNews(); ?
```



# Abstraction & Design Pattern for Structuring

```
class _NewsScreenState extends State<NewsScreen> {  
  late Future<List<Map>> news;  
  late AbstractNewsRepo newsRepo;  
  @override  
  void initState() {  
    super.initState();  
    newsRepo = AbstractNewsRepo.getInstance();  
    news = newsRepo.getNewsData();  
  }  
}
```

Is the news loaded from the Web ? Dummy data ? from a local DB ? from a Cache ?

This is not the concern of the this screen

